

LET - LLM Explanation Tool: Prototype Documentation

Abstract

The **LLM Explanation Tool (LET)** is a comprehensive web-based platform designed for generating, evaluating, and comparing natural language explanations from large language models (LLMs). Built for researchers and practitioners in explainable AI, LET addresses the growing need to understand and assess the quality of AI-generated explanations across multiple dimensions. While most existing explainability frameworks focus on feature attribution methods (e.g., LIME, SHAP), LET emphasizes **self-explanations** and **post-hoc explanations** expressed in natural language, reflecting the growing importance of LLMs in human-AI interaction.

LET enables multi-provider LLM integration (OpenAI, Gemini, DeepSeek, Groq, Ollama), supports traditional transformer classifiers (BERT) with SHAP-based explanations, and provides systematic evaluation of explanation quality using the LExT framework. The platform supports both interactive and batch processing of benchmark datasets, enabling side-by-side comparison of explanation types and providers.

This document provides comprehensive technical documentation for the LET prototype, including system architecture, supported models and datasets, evaluation methodology, user interface design, and installation instructions.

1. Introduction and Design Goals

1.1 Motivation and Context

The **LLM Explanation Tool (LET)** was designed to address the gaps identified in existing explainability frameworks and to provide a unified platform for generating, evaluating, and comparing natural language explanations from large language models (LLMs). While prior toolkits focus primarily on feature attribution methods such as LIME and SHAP, LET emphasizes **self-explanations** and **post-hoc explanations** expressed in natural language. This reflects the growing importance of LLMs in human-AI interaction and the need for explanations that are both **faithful** (accurately reflecting model reasoning) and **plausible** (convincing to human users).

Traditional explainability methods are designed for smaller models with fixed architectures and often produce token-level or feature-level importance scores. While these methods provide valuable insights into model behavior, they do not capture the narrative reasoning capabilities of modern LLMs, which can generate free-form textual explanations that resemble human justifications. LET bridges this gap by providing infrastructure for both traditional attribution-based explanations (via SHAP) and natural language explanations (via LLM prompting), enabling systematic comparison between these paradigms.

1.2 Design Requirements

The development of LET was guided by the following design goals and requirements:

Multi-provider Support

LET should support a wide range of LLMs, including commercial APIs (e.g., OpenAI, Gemini, DeepSeek, Groq) and locally deployed models (via Ollama). This ensures flexibility for both cloud-based experimentation and privacy-preserving offline use. The system must be provider-agnostic, meaning it connects to provider APIs rather than hardcoding specific models, ensuring the platform remains current as new models are released.

Integration of Traditional Classifiers

In addition to LLMs, LET supports smaller transformer-based models (e.g., BERT classifiers), which allow the generation of SHAP-based feature attribution explanations. This ensures continuity with traditional explainability approaches and provides baseline comparisons for evaluating LLM explanation quality.

Dataset Access and Flexibility

LET must provide seamless integration with benchmark datasets and allow users to choose between two processing modes: 1. **Batch classification and explanation generation** across an entire dataset 2. **Interactive exploration** of individual data points

Users should be able to upload custom datasets via CSV or import from Hugging Face Hub, ensuring the tool can be adapted to diverse experimental needs.

Dual Explanation Types

LET should enable both **self-explanations** (generated directly by LLMs in a single prompt) and **post-hoc explanations** (produced after predictions, potentially by a different model). This dual capability allows systematic comparison of explanation styles and their effects on trust calibration and user understanding.

Explanation Quality Metrics

LET must incorporate quantitative evaluation of explanation quality, particularly along the dimensions of **faithfulness** and **plausibility**, as conceptualized by the LExT framework. This ensures that explanations can be assessed beyond surface plausibility and enables measurement of how well explanations reflect actual model reasoning.

Unified Results Visualization

All model predictions, explanations, metrics, and user ratings should be summarized in an accessible results interface. This enables researchers and practitioners to easily compare models, datasets, and explanation types side by side, facilitating both qualitative and quantitative analysis.

Scalability and Usability

LET must remain lightweight, web-accessible, and suitable for integration into user studies. A modular architecture (backend, frontend, database) was chosen to ensure scalability and adaptability to different experimental setups. The system should support both small-scale interactive exploration and large-scale batch processing.

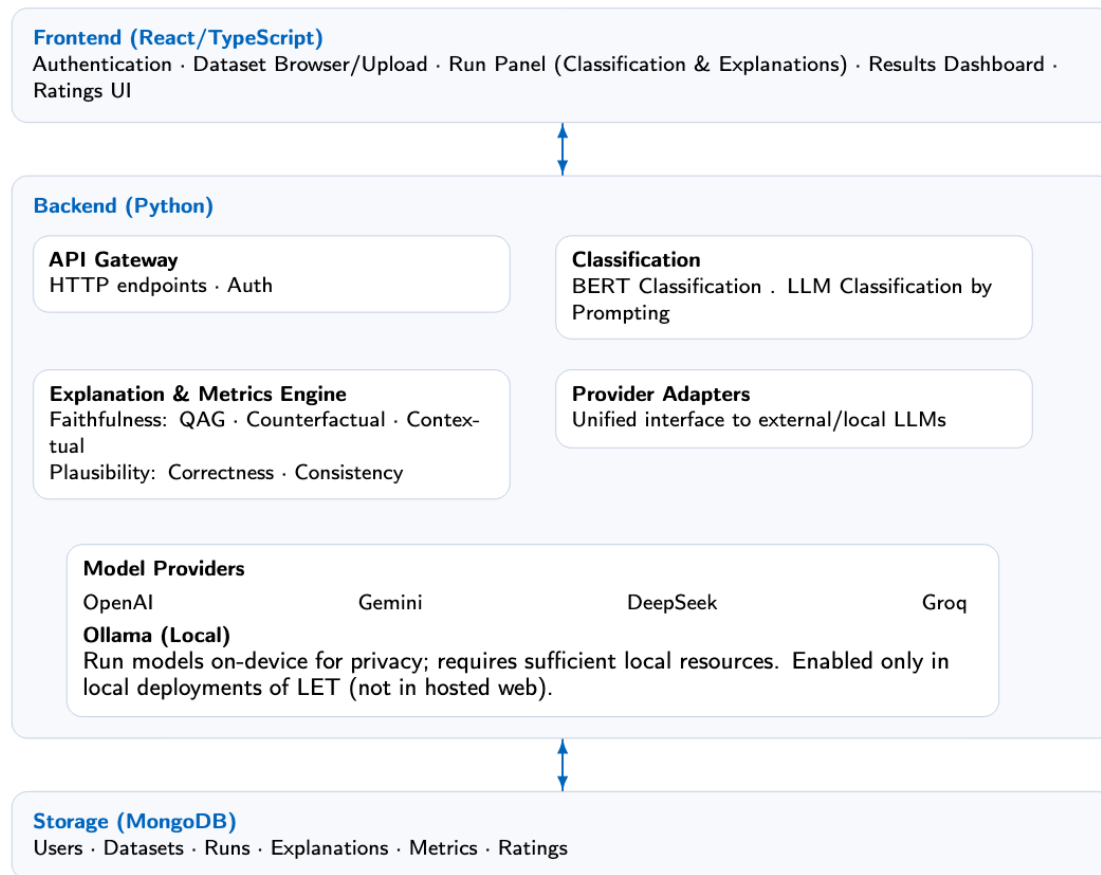
1.3 Contributions

By meeting these requirements, LET goes beyond existing explainability frameworks and establishes itself not only as a technical contribution but also as an experimental environment for studying the role of explanation faithfulness in human-AI decision making. The platform enables researchers to:

- Systematically generate explanations from multiple LLM providers
- Evaluate explanation quality using standardized faithfulness and plausibility metrics
- Compare self-explanations with post-hoc explanations and traditional attribution methods
- Collect human ratings and behavioral data on explanation effectiveness

2. System Architecture

LET follows a three-tier architecture ensuring modularity, scalability, and ease of use. The system consists of a Python-based backend, a React-based frontend, and a MongoDB database, allowing flexible integration of multiple models and datasets while supporting seamless user interaction.



LET System Architecture

Figure 1: LET system architecture with Model Providers integrated inside the Backend (Python) layer. The architecture follows a modular design where the frontend handles user interaction, the backend orchestrates computation and model inference, and the database persists results and configurations.

2.1 Backend (Python + Flask)

The backend is implemented in Python using the Flask framework. Python was chosen as the implementation language because it is the de facto standard for AI and machine learning research. Most modern deep learning frameworks (e.g., PyTorch, TensorFlow) and explainability libraries (e.g., SHAP, Captum) are natively supported in Python, making it the most practical and compatible choice for a system that integrates state-of-the-art models and evaluation techniques.

Within this ecosystem, Flask was selected as the web framework due to its lightweight, modular, and extensible design. Unlike more heavyweight frameworks such as Django, Flask imposes minimal architectural constraints, which allows for rapid prototyping and flexible integration of custom research components such as API adapters, explanation engines, and evaluation pipelines. Its simplicity in defining RESTful endpoints also provides

a clear and maintainable way to separate computation from presentation, ensuring that the backend can serve results efficiently to the frontend.

Backend Components

The Flask backend acts as the central hub for coordinating the following tasks:

API Gateway: - RESTful endpoints for authentication and task orchestration - Request validation and error handling - Session management and user authentication - Rate limiting and request queuing

Classification Engine: - Supports both LLM-based prompting and BERT classification - Unified interface for different model types - Handles both single-instance and batch processing - Manages confidence score computation and prediction formatting

Explanation & Metrics Engine: - Implements LExT evaluation framework - Computes faithfulness metrics: QAG, Counterfactual Stability, Contextual Faithfulness - Computes plausibility metrics: Correctness, Consistency - Supports metric computation for multiple datasets with different schemas - Aggregates metrics into trustworthiness scores

Provider Adapters: - Unified interface to external/local LLMs - Handles API authentication and request formatting - Manages rate limits and retries - Supports streaming responses where available - Normalizes outputs across different provider formats

In LET, the Flask backend integrates APIs from multiple commercial providers as well as locally hosted models through Ollama. In addition, smaller transformer-based models such as BERT are supported for traditional classification tasks and SHAP-based feature attribution explanations. Together, Python and Flask provide an optimal combination of research-friendly flexibility, ecosystem support, and deployment simplicity, making them well suited for the requirements of this platform.

2.2 Frontend (React + TypeScript)

The frontend is implemented in React with TypeScript, designed to provide an intuitive and interactive user interface. Users can register accounts, upload or select datasets, configure model providers, and initiate classification or explanation runs. Outputs are visualized in structured views, including prediction tables, explanation panels, faithfulness and plausibility scores, and user rating interfaces. Dedicated result pages enable side-by-side comparison across models and explanation types, while exploratory modes allow users to inspect samples one by one.

TypeScript was chosen over plain JavaScript because it enforces static typing, making the development process safer and reducing the likelihood of runtime errors in a complex system with many interconnected components. Explicit type definitions improve code readability and maintainability, which is especially beneficial for a research platform expected to evolve and integrate new features over time. Furthermore, React with

TypeScript is widely supported in the web development ecosystem, ensuring compatibility with UI libraries and testing frameworks.

Frontend Components

Authentication Module: - User registration and login - API key management interface - Session persistence and secure token storage

Dataset Browser/Upload: - Import from Hugging Face Hub - Local CSV file upload with validation - Dataset preview and metadata display - Management of multiple datasets

Run Panel (Classification & Explanations): - Model and provider selection - Batch size configuration - Chain-of-Thought prompting toggle (dataset-dependent) - Classification-only vs. classification-with-explanation modes - Previous run history and navigation

Results Dashboard: - Summary statistics (accuracy, F1, precision, recall) - Label distribution visualization - Predictions table with confidence scores - Highlighting of misclassifications - Export functionality for results

Ratings UI: - Per-instance explanation rating interface (1-5 scale) - Side-by-side comparison of explanation types - Rating submission and storage - Historical rating review

2.3 Database (MongoDB)

The database layer is built on MongoDB, chosen for its flexibility in storing semi-structured data such as model outputs, explanations, evaluation metrics, and user ratings. Each run is stored as a document containing metadata (e.g., dataset, model, provider), predictions, explanations, and computed scores. This document-oriented structure is well-suited for LET, since explanation data can vary in length and format across different models and tasks.

MongoDB was selected over traditional SQL-based databases (such as PostgreSQL or MySQL) because the data generated by LET is inherently heterogeneous. For example, explanations may be short rationales, multi-sentence narratives, or token-level attributions, and different providers can produce outputs with slightly different structures. Representing this variability in a relational schema would either require a highly normalized design—leading to complex queries and joins—or frequent schema modifications as the system evolves. By contrast, MongoDB’s JSON-like storage format naturally supports this variability and makes it straightforward to log and retrieve complete experiment runs.

Database Schema

Users Collection: - User credentials and profile information - API keys for different providers (encrypted) - User preferences and settings - Timestamp of registration and last login

Datasets Collection: - Dataset metadata (name, source, upload date) - Schema information (column names, data types) - Statistics (number of instances, class distribution) - Reference to stored data files or Hugging Face identifiers

Runs Collection: - Run metadata (user, dataset, model, provider, timestamp) - Configuration (batch size, Chain-of-Thought enabled, explanation type) - Performance metrics (accuracy, F1, precision, recall) - Reference to predictions and explanations

Predictions Collection: - Individual predictions with confidence scores - Ground truth labels - Input text and metadata - Explanation text (for self-explanations) - Faithfulness and plausibility scores - User ratings

Explanations Collection: - Generated explanations (self-explanation and post-hoc) - SHAP attributions (for BERT) - Evaluation metric breakdowns (QAG, Counterfactual, Contextual, Correctness, Consistency) - Timestamp and regeneration history

Ratings Collection: - User-provided ratings for explanations - Rating scale (1-5) - Timestamp and user identifier - Associated prediction and explanation references

Another advantage is scalability: MongoDB can efficiently handle both batch runs producing thousands of explanations and interactive exploration where individual queries need to be returned with low latency. Built-in indexing and flexible query capabilities further support filtering by dataset, model, or metric, which is essential for researchers analyzing large-scale experimental results. For these reasons, MongoDB provides the right balance between flexibility, performance, and maintainability for the LET platform.

2.4 Workflow Integration

The modular architecture ensures smooth integration between layers. The frontend sends requests to the backend, which performs computations and queries models or datasets. Results are stored in the database and returned to the frontend for visualization. This separation of concerns not only improves maintainability but also allows independent scaling of each component: more compute resources can be allocated to the backend for heavy model queries, while the frontend remains lightweight and responsive.

Typical Workflow:

1. User logs in via the frontend and configures API keys in Settings
2. User selects or uploads a dataset via the Dataset Management interface
3. User navigates to the Dataset View and selects classification mode (with or without explanation)
4. Frontend sends a request to the backend with run configuration
5. Backend orchestrates model inference via Provider Adapters
6. Classification Engine processes predictions and formats results
7. Explanation & Metrics Engine computes faithfulness/plausibility scores
8. Results are stored in MongoDB and returned to the frontend

- 9. Frontend renders the Classification Dashboard with summary statistics
- 10. User navigates to individual instances via the Explanation Page
- 11. User can regenerate explanations, view metrics, and provide ratings
- 12. All interactions are logged to the database for reproducibility

This modular and provider-agnostic design allows LET to serve dual purposes: (1) as a practical tool for researchers and practitioners to test and compare explanations across multiple LLMs, and (2) as an experimental platform for conducting user studies.

3. Supported Providers and Models

A key design goal of LET is flexibility: the platform is not tied to a fixed set of models but is instead designed to integrate with any large language model that is exposed through an API. This approach ensures that the system remains extensible and future-proof, since new models can be seamlessly incorporated as soon as they become available from a provider. The only requirement for users is to obtain valid API keys from the provider of their choice. Once authenticated, users are free to access the full range of models offered by that provider, including different versions, parameter sizes, or specialized variants. In this way, LET avoids the limitations of static benchmarking platforms and adapts naturally to the rapidly evolving landscape of language models.

3.1 Provider Overview

Table 1 provides an overview of the providers currently supported within LET. Most of these are commercial cloud-based platforms that continuously release new and improved language models, such as **OpenAI**, **Google DeepMind’s Gemini**, **DeepSeek**, and **Groq**. Since LET is designed to connect directly with providers rather than fixed models, users can freely select any available model from these platforms once they have access credentials. This means that LET automatically benefits from the rapid pace of development in the field: if a provider introduces a more powerful model, it becomes immediately usable within the tool without requiring architectural changes.

Provider / Model Type	Example Models	Deployment Type
OpenAI	GPT-3.5, GPT-4, GPT-4o	Cloud-based
Gemini (Google DeepMind)	Gemini 1.5 Pro, Gemini Ultra	Cloud-based
DeepSeek	DeepSeek LLMs	Cloud-based
Groq	High-speed inference-optimized LLMs	Cloud-based
Ollama	Llama 2, Mistral, custom local models	Local (on-device)
Baseline Transformers	BERT (with SHAP	Built-in / Local

Provider / Model Type	Example Models explanations)	Deployment Type
-----------------------	---------------------------------	-----------------

Table 1: Currently supported providers and models in LET. Cloud-based providers require API keys obtained from the respective platforms. Ollama enables local deployment of large language models on a user’s own machine, but is only supported in the local version of LET. In addition, LET includes built-in support for baseline transformer models such as BERT, with SHAP-based explanations to provide interpretable comparisons against modern LLMs.

3.2 Cloud-Based Providers

OpenAI

OpenAI provides access to the GPT family of models, including GPT-3.5, GPT-4, and GPT-4o. These models are widely used in research and production settings and offer strong performance across diverse tasks. LET integrates with OpenAI’s API using user-provided API keys, allowing access to any model available through the platform at the time of use.

Gemini (Google DeepMind)

Google’s Gemini models represent state-of-the-art multimodal capabilities, though LET currently uses them only for text-based tasks. The Gemini API provides access to models such as Gemini 1.5 Pro and Gemini Ultra, which offer competitive performance on reasoning and explanation generation tasks.

DeepSeek

DeepSeek provides specialized LLMs optimized for reasoning tasks. Integration with DeepSeek follows the same pattern as other cloud providers, requiring users to supply API credentials.

Groq

Groq specializes in high-speed inference infrastructure, offering dramatically faster response times compared to traditional cloud APIs. This makes Groq particularly suitable for interactive exploration and rapid iteration during explanation generation. The trade-off is that model selection may be more limited compared to larger providers.

3.3 Local Deployment (Ollama)

In addition to cloud-based providers, LET also supports **Ollama**, a tool designed for running large language models locally on the user’s own machine. Ollama enables privacy-preserving experimentation by avoiding external API calls and storing all computation and data locally. This is particularly appealing for sensitive use cases where data cannot leave the user’s machine, or for researchers who wish to test models that may not be publicly available through cloud services.

However, running models locally introduces new challenges: larger models require substantial computational resources, including GPUs with high memory capacity, and can be slow or infeasible to run on consumer-grade hardware. As such, LET provides Ollama integration with the explicit caution that users must ensure their system is capable of supporting the model size they intend to run.

Important Note: Because LET is primarily designed to be deployed as a hosted web application, Ollama integration is **only available in local deployments** of the tool. That is, users who clone the repository and run LET on their own machine can enable Ollama support, whereas the hosted version of LET does not include this functionality due to resource constraints and infrastructure requirements.

3.4 Baseline Transformers (BERT with SHAP)

Beyond large-scale providers, LET also incorporates smaller transformer-based models for baseline comparison. In particular, **BERT** is supported as a classifier for tasks such as sentiment analysis and domain-specific classification. While BERT does not natively generate free-form natural language explanations, LET complements it with **SHAP**-based feature attribution methods. These highlight the contribution of individual input tokens to the model's prediction, providing a transparent explanation of decision-making.

Furthermore, LET extends this functionality by allowing users to pass SHAP attributions to a large language model, which can then generate a natural language explanation grounded in the highlighted features. This hybrid setup combines the interpretability of classical attribution methods with the expressiveness of modern LLMs, offering a richer and more flexible explanation pipeline.

3.5 Provider-Agnostic Design Philosophy

By supporting both cloud-based and local providers, as well as smaller transformer baselines, LET balances accessibility and control. Cloud-based APIs are convenient, scalable, and do not require local resources, but they involve recurring costs, dependency on third-party availability, and potential privacy concerns. Local deployment with Ollama offers the opposite trade-off: full control and privacy, but at the cost of requiring significant hardware capacity. Meanwhile, smaller models like BERT with SHAP explanations provide interpretable baselines that help contextualize the capabilities of modern LLMs.

LET's architecture is designed to accommodate all three approaches, giving users the freedom to choose the setup that best matches their resources, needs, and research goals. This dual strategy reflects the central philosophy of LET: to provide a flexible, extensible, and user-centered environment for exploring the faithfulness and plausibility of large language model explanations.

4. Supported Datasets

LET is designed to support a diverse range of datasets that span multiple domains and explanation styles, making it possible to evaluate models under varied conditions. The included datasets were selected to cover tasks in sentiment analysis, legal reasoning, biomedical question answering, commonsense reasoning, sarcasm detection, and deception detection. Some provide ground-truth human explanations, which enable plausibility assessment, while others lack gold rationales, making them suitable for isolating faithfulness effects.

4.1 Dataset Overview

Table 2 provides a comprehensive overview of the datasets supported in LET, including their domain, task type, label space, availability of gold explanations, and example use cases.

Dataset	Domain	Task	Labels	Gold Explanations	Use Case
IMDB	Movies	Sentiment Analysis	positive / negative	X	Faithfulness evaluation on long-form text
CaseHOLD	Legal	Case Law Reasoning (MCQ)	correct / distractors	X	Domain-specific reasoning in high-stakes contexts
PubMedQA	Biomedical	Question Answering	yes / no / maybe	✓	Medical reasoning with gold rationales
ECQA	Common sense	QA with Explanations	multiple choice	✓	Plausibility assessment with human references
SNARKS	Social Media	Sarcasm Detection	sarcastic / not sarcastic	X	Pragmatic reasoning and tone analysis
Deceptive Opinion	Reviews	Deception Detection	truthful / deceptive	X	Primary dataset for user study

Table 2: Datasets supported in LET. Examples are included for illustration purposes. Some datasets provide gold human explanations (✓), enabling plausibility evaluation, while others (X) are used primarily for faithfulness assessment.

4.2 Dataset Descriptions and Rationale

IMDB Sentiment

Task: Binary sentiment classification of long-form movie reviews (positive vs. negative).

Why included: IMDB is one of the most widely used benchmarks for text classification, and its large size and long-form inputs make it particularly suitable for evaluating explanation methods in more complex, real-world settings. Unlike short review datasets such as Deceptive Opinion, IMDB reviews often span multiple sentences or even paragraphs, which forces models to process and reason over nuanced linguistic structures, mixed sentiment, and contextual cues.

Since no gold human-written explanations are available, the dataset cannot be used directly for plausibility evaluation. Instead, it serves primarily as a benchmark for assessing **faithfulness**, where explanations must identify which specific words, phrases, or passages drive the model’s sentiment classification. In the context of LET, IMDB allows researchers to test whether attribution-based methods such as SHAP, or LLM-generated self-explanations, accurately reflect the underlying reasoning used by the model when dealing with longer and noisier input texts. This makes IMDB an important complement to smaller, domain-specific datasets, as it challenges both models and explanation methods to remain faithful under more naturalistic, unconstrained input conditions.

CaseHOLD

Task: Multiple-choice legal reasoning, where the model is presented with a case fact description and must select the correct holding statement.

Why included: CaseHOLD is designed to evaluate domain adaptation in the legal domain, which requires structured and high-stakes reasoning. Unlike datasets such as ECQA, it does not provide gold human-written explanations. This limitation makes plausibility evaluation less feasible, but it provides an ideal setting for studying **faithfulness**, since explanations must capture the legal reasoning process rather than mimic a reference text.

In LET, CaseHOLD enables users to test whether LLM-generated explanations align with the logic of selecting the correct holding in complex legal scenarios. By comparing models across providers, users can explore whether explanation quality scales with model size or architecture in specialized domains. Despite the absence of gold rationales, CaseHOLD is valuable for assessing the consistency, domain-specific alignment, and faithfulness of generated explanations in legal reasoning tasks.

PubMedQA

Task: Biomedical yes/no/maybe question answering based on PubMed abstracts.

Why included: PubMedQA evaluates domain-specific reasoning in healthcare, a high-stakes field where explanation quality is particularly important. Each instance typically includes a clinical or biomedical question together with an abstract retrieved from PubMed, which contains contextual information about the patient, study design, or medical intervention that may not be explicitly stated in the question. This separation between question and context makes the dataset valuable for testing whether models generate explanations that truly integrate external evidence rather than relying only on surface cues. Gold rationales are provided, making it possible to measure plausibility of the explanations.

PubMedQA has also been adopted in recent explanation evaluation work, such as the LExT framework, where it was used to assess both faithfulness and plausibility of model-generated explanations. Its combination of domain-specific complexity, gold-truth rationales, and context-dependent reasoning makes PubMedQA a critical dataset for analyzing how well LLM explanations align with biomedical evidence while remaining comprehensible to human evaluators.

ECQA (Explanations for Commonsense Question Answering)

Task: Commonsense multiple-choice question answering with human-provided natural language explanations.

Why included: ECQA is one of the few datasets that explicitly provides gold free-text explanations alongside answer labels. This makes it a strong benchmark for plausibility evaluation, since model-generated explanations can be directly compared against human rationales. Beyond plausibility, the dataset also allows for analyzing faithfulness by checking whether the explanations highlight the reasoning steps actually needed to select the correct answer.

In LET, ECQA is particularly valuable because it supports both self-explanations from LLMs and the integration of **chain-of-thought (CoT) prompting**. Users can choose to enable CoT, prompting models to generate step-by-step reasoning, or to run without it. This flexibility makes it possible to compare plausibility and faithfulness across different prompting styles, providing deeper insight into how explanation format influences evaluation. Together, the commonsense focus and availability of gold rationales position ECQA as a key dataset for studying explanation quality in LLMs.

SNARKS

Task: Sarcasm detection in short conversational or social media text.

Why included: Sarcasm detection is a notoriously difficult NLP task because it requires going beyond surface-level semantics to capture pragmatic cues such as tone, intent, and contextual mismatch. An utterance may look positive on the surface (e.g., “Great, another Monday morning meeting...”) while actually conveying a negative sentiment. For this reason, explanations in sarcasm detection are especially valuable: a faithful explanation should highlight why an apparently literal statement is actually sarcastic, pointing to subtle linguistic signals, incongruity, or context dependence.

SNARKS serves as a challenging testbed for explanation methods, since models often rely on spurious correlations or superficial lexical cues rather than genuinely capturing pragmatic reasoning. Unlike datasets such as ECQA or PubMedQA, it does not include gold human-provided explanations, which means plausibility cannot be directly evaluated. Instead, the dataset is mainly useful for analyzing **faithfulness**—whether model-generated explanations truly reflect the reasoning behind sarcasm predictions. Within LET, SNARKS allows researchers to compare how different LLMs and attribution methods handle pragmatically complex phenomena, offering insight into whether explanations meaningfully capture higher-level reasoning about context and intent.

Deceptive Opinion

Task: Binary classification of hotel reviews as deceptive vs. truthful.

Why included: This dataset consists of relatively short, realistic reviews with clear binary labels, making it highly suitable for controlled experiments on explanation quality. Unlike datasets such as ECQA or PubMedQA, it does not include gold human-written explanations, which means plausibility cannot be directly measured. Instead, the focus is on evaluating **faithfulness**, since explanations must highlight linguistic or stylistic cues of deception that justify the classification.

In this thesis, Deceptive Opinion serves as the **primary dataset for the user study**. Its balanced design and straightforward task make it ideal for systematically testing how varying levels of explanation faithfulness affect user reliance, confidence, and decision-making behavior. Importantly, the dataset is known to be highly challenging for humans—studies have shown that unaided human accuracy is often no better than random guessing. This characteristic is essential for experimental setups requiring situations where users initially disagree with the AI. The inherent difficulty of the task ensures a sufficient number of such disagreements, enabling meaningful measurement of appropriate reliance metrics. By pairing faithful and unfaithful explanations with both correct and incorrect predictions, the dataset provides a clean and controlled setting for studying whether faithfulness improves calibrated reliance or, conversely, whether unfaithful explanations risk misleading participants.

4.3 Custom Dataset Upload

In addition to the built-in benchmark datasets, LET supports custom dataset uploads via two mechanisms:

1. **Hugging Face Hub Integration:** Users can import datasets directly from Hugging Face by providing the dataset identifier (e.g., `imdb`, `pubmed_qa`). This ensures standardized formatting and compatibility with LET's evaluation pipelines.
2. **Local CSV Upload:** Users can upload their own datasets in CSV format. The system validates the schema and maps columns to expected fields (input text, label, choices, etc.). This enables experimentation with domain-specific or proprietary

datasets while maintaining compatibility with LET's explanation and evaluation infrastructure.

Custom datasets extend the flexibility of LET beyond controlled benchmark experiments, making the tool adaptable for exploratory research in diverse domains.

5. Prompt Design and Explanation Modes

A central design aspect of LET is the use of flexible prompts that adapt to the task and dataset while maintaining comparability across models. For each supported dataset, prompts are constructed to capture the task definition (e.g., sentiment classification, question answering, or legal reasoning) and to elicit explanations in a consistent format. LET supports multiple modes of explanation generation, each serving different research and practical goals.

5.1 Explanation Modes

LET supports three complementary explanation approaches:

5.1.1 Self-Explanation (LLM-native)

The model generates both prediction and explanation in a single prompt, capturing its internal reasoning directly in natural language. This mode is the primary focus of LET's evaluation framework, as it allows direct assessment of both faithfulness and plausibility. Self-explanations reflect the model's reasoning process as expressed during prediction, making them suitable for metrics such as QAG (Question-Answer Generation) and Counterfactual Stability.

Characteristics: - Prediction and rationale generated simultaneously - Reflects model's reasoning at inference time - Suitable for faithfulness and plausibility evaluation - Requires careful prompt design to ensure consistent format

5.1.2 Post-Hoc Explanation

Classification and explanation are decoupled: one model generates predictions, then the same or a different model explains the decision afterward. This approach provides flexibility for cross-model analysis, where a smaller model performs classification and a larger model provides explanations. Post-hoc explanations can also incorporate additional information not available during prediction, such as SHAP attributions.

Characteristics: - Prediction and explanation generated separately - Enables cross-model explanation analysis - Can incorporate attribution-based features (e.g., SHAP) - Not suitable for standard faithfulness metrics (explanation does not reflect predictor's reasoning)

5.1.3 SHAP-Augmented Explanation

For BERT classifiers, token-level SHAP attributions are computed and either: - Visualized directly as feature importance - Verbalized into natural language by an LLM

This hybrid approach combines statistical attribution with narrative reasoning, bridging traditional feature-based explainability with modern natural language explanations.

Characteristics: - Combines feature attribution with natural language - Provides both local (token-level) and global (narrative) interpretability - Allows comparison between attribution scores and LLM rationales - Suitable for smaller transformer models (BERT)

5.2 Chain-of-Thought (CoT) Prompting

For datasets like ECQA, users can enable CoT to elicit step-by-step reasoning. CoT prompting has been shown to improve reasoning performance by encouraging models to break down complex problems into intermediate steps. In LET, CoT is offered as an optional configuration for datasets where multi-step reasoning is beneficial.

When CoT is Available: - ECQA (commonsense reasoning with multiple choices) - CaseHOLD (legal reasoning with multiple holdings) - PubMedQA (biomedical reasoning with contextual passages)

When CoT is Not Applicable: - IMDB (sentiment analysis is typically single-step) - Deceptive Opinion (short reviews with binary classification) - SNARKS (sarcasm detection relies on implicit cues)

5.3 Role Prompting Strategy

An additional design choice in LET is the use of **role prompting** in many of the dataset-specific instructions. Role prompting encourages the model to adopt a specific perspective or domain-relevant role when generating its predictions and explanations. This has been shown to improve zero-shot reasoning performance by aligning the model's responses with the expectations and reasoning style of that role, making explanations more coherent and contextually appropriate.

Examples of Role Prompting: - CaseHOLD: "Assume you are a legal advisor..." - PubMedQA: "Assume you are a medical advisor..." - Deceptive Opinion: "You are a deceptive hotel review detection system..." - IMDB: "Assume you are a movie critic..."

In practice, role prompting helps constrain the output space of the model, reduces off-task or generic answers, and increases the plausibility of the generated explanations by embedding them in a domain-specific narrative. For example, in PubMedQA, framing the task as medical advice not only yields more medically grounded predictions but also produces explanations that resemble the style of professional reasoning.

5.4 Prompt Templates

Below are representative prompt templates for selected datasets. The complete set of prompts for all supported datasets (including CaseHOLD, PubMedQA, and SNARKS) is available in the LET codebase repository.

5.4.1 IMDB Sentiment (Post-hoc SHAP-Augmented Explanation)

This prompt is used when a BERT classifier has already produced a sentiment prediction and SHAP has identified the most important tokens. An LLM is then asked to verbalize these attributions into a natural language explanation.

Assume you are a movie critic.

Explain this sentiment analysis result in simple terms with most affecting words provided by SHAP:

Text: {text}
Sentiment: {label} ({score}% confidence)

SHAP:
{shapwords}

Focus on key words and overall tone.
Keep explanation under 3 sentences.

Note: Confidence scores are explicitly reported because the underlying model is a BERT classifier. This makes the prediction more transparent and provides additional context for users when interpreting SHAP-based explanations.

5.4.2 ECQA Self-Explanation and Classification (without CoT)

This prompt asks the model to select the best answer from five options and provide a brief explanation in a single response.

Given the following question and five answer options, select the best answer and explain your choice in 2-3 sentences. YOU MUST ONLY CHOOSE ONE OF THE CHOICES.

Question: {question}

Choices:
{choices[0]}
{choices[1]}
{choices[2]}
{choices[3]}
{choices[4]}

Format your answer as:

Answer: <Your Choice>

Explanation: <your explanation here>

5.4.3 ECQA Self-Explanation and Classification (with CoT)

This prompt explicitly requests step-by-step reasoning before the final answer, implementing chain-of-thought prompting.

You are solving a commonsense multiple-choice question.

First, think through the problem step by step, considering why each option may or

may not be correct. Then state the final answer clearly.

Question: {question}

Choices:

{choices[0]}

{choices[1]}

{choices[2]}

{choices[3]}

{choices[4]}

Format your response as:

Explanation: <step by step reasoning, a few sentences>

Answer: <Your Choice>

5.4.4 Deceptive Opinion Self-Explanation and Classification

This prompt is used for the primary user study dataset, requesting both a prediction and a brief explanation.

You are a deceptive hotel review detection system. You will choose "truthful" or "deceptive" as your answer and explain your decision in 2-3 sentences.

Question: {question}

Format your answer as:

Answer: <Choice as "truthful" or "deceptive">

Explanation: <your explanation here>

5.4.5 Deceptive Opinion Post-Hoc Explanation

This prompt is used when a classification has already been made (either by an LLM or BERT), and a separate explanation is requested afterward.

You are a deceptive hotel review detection system.

Explain this hotel review authenticity detection result in simple terms:

Review: {text}
Prediction: {label} ({score}% confidence)

Focus on the indicators of authenticity or deception.
Keep explanation under 3 sentences.

5.5 Classification-Only Prompts

In addition to explanation prompts, LET also supports classification-only prompts. These are structurally identical to the explanation prompts but with the explanatory component removed, instructing the model to output only the predicted label. This design ensures consistency across tasks and datasets, while allowing direct comparison between pure classification and classification-with-explanation modes. Because the difference lies solely in stripping out the explanation requirement, full listings of these prompts are omitted for brevity but follow the same structure as shown above.

5.6 Design Principles for Prompt Construction

When designing prompts for LET, the following principles were followed:

1. **Consistency:** All prompts follow a standardized format (task description, input, output format) to ensure comparability across datasets and models.
2. **Explicit Formatting:** Prompts explicitly specify the expected output format (e.g., “Answer: ... Explanation: ...”) to facilitate parsing and evaluation.
3. **Length Constraints:** Explanations are typically constrained to 2-3 sentences to ensure conciseness and focus, while allowing sufficient detail for evaluation.
4. **Role Alignment:** Role prompts align the model with domain-specific expectations, improving the quality and appropriateness of generated explanations.
5. **Flexibility:** Prompts are designed to work across multiple providers and model sizes, avoiding provider-specific optimizations that would reduce generalizability.

6. Faithfulness and Plausibility Evaluation

To systematically evaluate the quality of natural language explanations, LET adopts the **LExT (LLM Explanation Trustworthiness)** framework. LExT conceptualizes explanation quality as a function of two complementary dimensions: **faithfulness** and **plausibility**. An explanation is considered trustworthy only if it satisfies both criteria. Plausibility reflects how convincing and human-like an explanation appears, while faithfulness measures whether the explanation accurately represents the underlying reasoning process of the model. If only one dimension is satisfied, explanations risk being either deceptively plausible but unfaithful, or faithful but unconvincing to users. This duality makes

faithfulness and plausibility indispensable for a robust evaluation of explanation quality in large language models (LLMs).

Important Note: Both faithfulness and plausibility can only be applied to **self-explanations**, where the model simultaneously produces a prediction and its rationale in the same output. This is because faithfulness requires probing whether the explanation aligns with the model’s actual reasoning during prediction, and plausibility requires comparing the generated rationale against human-written or ground-truth references. In contrast, **post-hoc explanations** (e.g., when an explanation is generated after a prediction, or when one model explains the output of another) do not directly reflect the reasoning of the predicting model, but rather provide an external justification. Therefore, in LET, faithfulness and plausibility metrics are restricted to self-explanations, while post-hoc explanations are offered for user interpretability and qualitative comparison.

6.1 Plausibility

Plausibility is operationalized in LExT through two metrics: **correctness** and **consistency**. These metrics are designed to capture semantic alignment with human-authored ground-truth explanations as well as the stability of generated explanations under repeated or paraphrased queries.

6.1.1 Correctness

Correctness measures how closely a generated explanation aligns with a human-annotated or expert-provided ground truth. The first step is to embed the predicted explanation ($Pred$) and the ground-truth explanation (GT) using BERT embeddings, and then compute cosine similarity:

$$\text{Accuracy} = \cos(\text{BERT}(GT), \text{BERT}(Pred))$$

However, embedding similarity alone can overestimate quality by giving high scores to irrelevant or partial explanations. To mitigate this, LExT introduces **Named Entity Recognition (NER) weighting**, which emphasizes overlap in domain-specific terms such as diseases, drugs, or clinical findings in medical datasets. Entities are extracted using a fine-tuned DeBERTaV3 MedNER model, and the overlap is scaled using an exponential factor $\beta = 0.2$:

$$\text{NER Weight} = \left(\frac{|NER_{GT} \cap NER_{Pred}|}{|NER_{Pred}|} \right)^\beta$$

The final correctness score is then given by:

$$\text{Correctness} = \text{Accuracy} \times \text{NER Weight}$$

This ensures that explanations are rewarded for capturing critical domain-specific content and penalized for including irrelevant or hallucinated information.

Implementation Note: In LET, the NER-weighted variant is applied only to the medical dataset (PubMedQA), where high-quality domain-specific NER models exist. For all other datasets, correctness is computed solely using BERT-based cosine similarity between predicted and reference explanations.

6.1.2 Consistency

Consistency evaluates whether a model produces stable explanations when given semantically equivalent inputs. It is assessed using two complementary methods:

Iterative Stability:

The same input is provided to the model multiple times. Explanations are compared with the ground truth using cosine similarity, and the variance is calculated:

$$\text{Iterative Stability} = 1 - \text{Var}(\cos(GT, \text{Pred}_i))$$

Low variance indicates that explanations are stable across iterations.

Paraphrase Stability:

The input is paraphrased into several equivalent forms, and explanations are generated for each version. The variance across these outputs is then computed. Models with high paraphrase stability generate consistent explanations regardless of input phrasing, while unstable models produce divergent or generic responses.

Together, correctness and consistency ensure that plausibility evaluation goes beyond surface-level similarity, capturing both factual alignment and robustness.

6.2 Faithfulness

Faithfulness is concerned with whether an explanation genuinely reflects the reasoning process of the model. LExT introduces three complementary tests—**Question-Answer Generation (QAG)**, **Counterfactual Stability**, and **Contextual Faithfulness**—to probe this property.

6.2.1 Question-Answer Generation (QAG)

This metric tests whether an explanation contains sufficient information to answer auxiliary questions derived from it. A larger LLM generates a set of questions based on the predicted explanation, which are then answered by the target model using the explanation alone. If the explanation provides enough reasoning to answer the derived questions correctly, it is considered faithful.

$$\text{QAG Score} = \frac{\text{\#Positive Answers}}{\text{\#Total Questions}}$$

High QAG scores indicate that explanations are self-consistent and reflect the model's prediction logic.

Implementation Details: - A more capable LLM (e.g., GPT-4) generates 3-5 auxiliary questions from the explanation - The target model attempts to answer each question using only the explanation as context - Answers are evaluated for correctness (binary: correct/incorrect) - The final score is the fraction of correctly answered questions

6.2.2 Counterfactual Stability

Counterfactual tests probe whether explanations remain faithful when deliberately contradicted. The explanation is rephrased to imply the opposite label (e.g., “Yes” → “No”), and the model is re-evaluated under this counterfactual explanation. Scoring is defined as:

$$\text{Counterfactual Stability} = \begin{cases} +1 & \text{if prediction flips correctly,} \\ 0 & \text{if the model fails to respond meaningfully,} \\ -1 & \text{if the model repeats its initial prediction} \end{cases}$$

This score is then normalized to the range $[0,1]$. Faithful explanations should allow the model to adapt its answer when the rationale is flipped.

Implementation Details: - The original explanation is rephrased to support the opposite conclusion - For binary tasks: flip between the two labels (positive/negative, truthful/deceptive) - For multiple-choice tasks: rephrase to support a different option - The model is re-prompted with the counterfactual explanation - A faithful explanation should cause the model to change its prediction accordingly

6.2.3 Contextual Faithfulness

This test evaluates whether explanations depend on the input context. The model is first asked to identify the most important tokens or phrases that justify its decision. These tokens are then systematically **redacted** in two phases:

Complete Redaction:

All critical tokens are removed simultaneously. A faithful explanation should fail or report insufficient information.

Sequential Redaction:

Tokens are removed one by one, and the fraction of “Unknown” responses is recorded.

The contextual faithfulness score is then given by:

$$\text{Contextual Faithfulness} = \frac{\text{\#Unknown Responses}}{\text{\#Total Prompts}}$$

This metric penalizes explanations that appear confident even when key context is removed, a hallmark of unfaithfulness.

Implementation Details: - Model identifies 5 most important words/phrases for its decision - Phase 1: All 5 are redacted; model should respond “Unknown” or “Insufficient information” - Phase 2: Words are added back one at a time; measure at what point model

becomes confident - Higher scores indicate stronger dependence on identified context - Unfaithful explanations remain confident even after critical information is removed

6.3 The LExT Score: Combining Faithfulness and Plausibility

Finally, LExT aggregates plausibility (correctness + consistency) and faithfulness (QAG + counterfactual + contextual) into a unified trustworthiness score. Each component is min-max scaled, and the aggregated score is calculated as a weighted harmonic mean to ensure that deficiencies in one dimension cannot be compensated by strength in another.

This holistic framework highlights that:

- Explanations that are **plausible but unfaithful** may deceive users by sounding convincing while misrepresenting reasoning.
- Explanations that are **faithful but implausible** may be ignored or distrusted by users.
- Only explanations that are both **faithful and plausible** can be considered trustworthy.

In LET, this framework is integrated to systematically evaluate explanations. While both faithfulness and plausibility are supported in the tool, research applications may focus primarily on faithfulness when gold rationales are unavailable, or when studying the effects of explanation quality on human-AI interaction.

6.4 Adaptations for Diverse Datasets

While the original LExT framework was implemented for PubMedQA and QPain, both biomedical QA datasets with rich contextual passages and gold-standard rationales, the datasets used in LET (IMDB, Deceptive Opinion, CaseHOLD, ECQA, SNARKS) differ substantially in structure and label space. To extend LExT beyond biomedical QA, LET modifies both the prompts and evaluation logic of the faithfulness metrics while preserving their conceptual foundations.

Counterfactual Stability Adaptations

In the original LExT setup, explanations for PubMedQA were rephrased to support the opposite answer (yes, no, or maybe). For other datasets, counterfactual prompts are rewritten to match their label spaces:

- **IMDB Sentiment:** Flip between positive and negative
- **Deceptive Opinion:** Flip between truthful and deceptive
- **SNARKS:** Flip between answer options (A) and (B)
- **CaseHOLD:** Select a different random holding from the list of candidate legal statements
- **ECQA:** Rephrase to support a different answer choice

These modifications ensure that counterfactual rephrasing produces a valid contradictory rationale regardless of the dataset format.

Question-Answer Generation (QAG) Adaptations

LEXT originally generated auxiliary biomedical questions from an explanation and checked whether they could be answered consistently. LET adapts this to other domains by tailoring the type of questions generated:

- **IMDB:** New movie reviews are generated and checked for consistency with sentiment explanations
- **Deceptive Opinion:** New hotel reviews are generated to test whether explanations supported deceptive/truthful classification
- **CaseHOLD:** Legal scenarios and holdings are generated
- **SNARKS:** Sarcasm-related cues are used
- **ECQA:** Commonsense scenarios are generated based on the explanation

This keeps the spirit of QAG intact—probing whether explanations provide enough reasoning to support similar cases—while adjusting the surface form to match dataset-specific tasks.

Contextual Faithfulness Adaptations

In PubMedQA, contextual faithfulness involves redacting critical tokens from the passage and testing whether the model can still answer. LET extends this in two ways:

- For datasets without long contexts (IMDB, Deceptive Opinion, SNARKS), the model is first asked to extract the **five most important words or phrases** from the explanation or review. These are then redacted, and the model’s ability to reproduce the prediction is tested.
- A two-stage evaluation is applied: if the model fails with complete redaction, words are then added back one at a time to measure whether the explanation is truly anchored in those tokens.
- For CaseHOLD, prompts emphasize legal concepts, facts, and reasoning elements to align with domain-specific interpretability.

This generalization allows contextual faithfulness to be applied even in tasks where no explicit “context passage” is available.

6.5 Summary

The LEXT framework provides a comprehensive methodology for evaluating explanation quality across two essential dimensions. In LET, these metrics are automatically computed for self-explanations and displayed in the user interface, enabling researchers to systematically assess and compare explanation methods. The adaptations for diverse

datasets ensure that the evaluation methodology remains conceptually grounded while being practically applicable across sentiment analysis, legal reasoning, biomedical QA, commonsense reasoning, sarcasm detection, and deception detection tasks.

7. User Interfaces and Workflow

While the backend and architectural components of LET provide the computational foundation, the **user interface** defines how researchers and participants interact with the system. The frontend is designed to be task-oriented, guiding users through a structured workflow from dataset selection to explanation evaluation. This section outlines the key interfaces and the intended workflow.

7.1 Registration Screen

The registration screen allows new users to create an account in LET. In addition to providing basic account information (username, email, password), users can also configure their connection to external language model providers during registration. The right-hand panel offers optional fields to enter API keys for supported providers such as OpenAI, Groq, DeepSeek, OpenRouter, and Gemini.

This design has two benefits. First, it enables users to begin experimenting with models immediately after account creation, without the need to separately configure API access later. Second, the flexibility of adding keys during registration or later in the settings panel ensures that LET accommodates both novice users (who may skip this step initially) and advanced users (who may want to preconfigure all providers).

The integration of API key management into the registration process reflects LET's core principle of being **provider-agnostic**: users decide which models to use by linking their own API credentials, ensuring both flexibility and compliance with provider terms of service.

Create Account

 Account Information

Username

Email address

Password

Create Account

Already have an account? [Login here](#)

 API Keys (Optional)

 To use Large Language Models for explanations and classifications, you need at least one API key available. You can add these later in your settings.

OpenAI API Key

Grok API Key

DeepSeek API Key

OpenRouter API Key

Gemini API Key

Registration Screen

Figure 2: Registration interface of LET, showing both account information fields and optional provider API key configuration.

7.2 Login Screen

The login screen provides a simple entry point into LET. Users can log in with their credentials (email and password) to access the system. Authentication ensures that runs, ratings, and results are linked to individual user accounts, allowing reproducibility and personalized tracking of experiments.

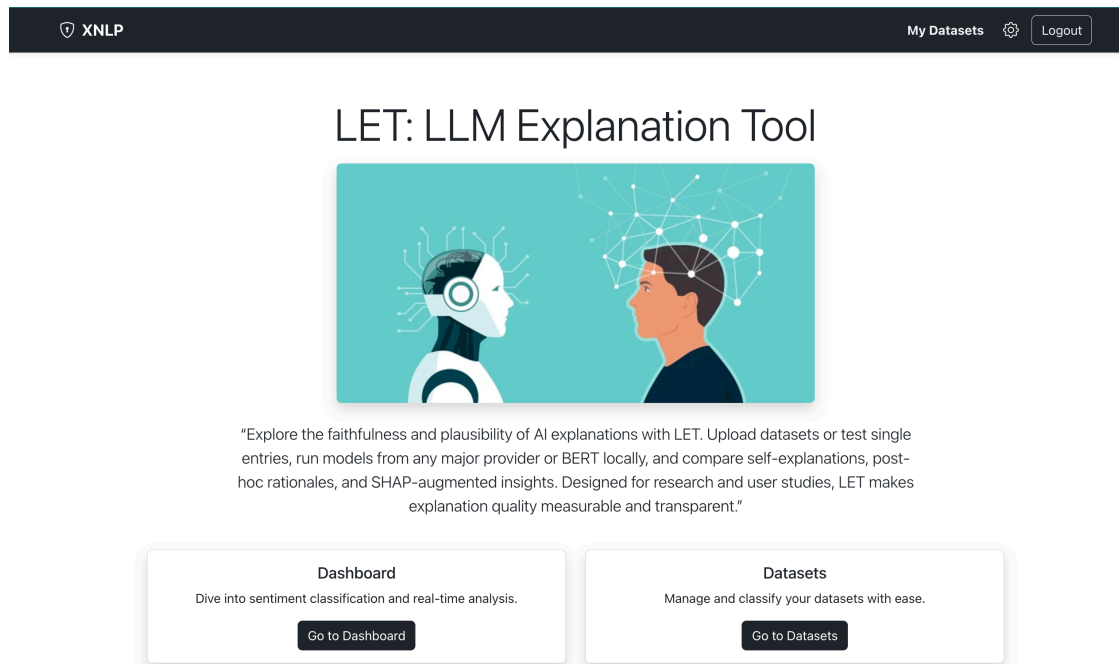
The image shows a login interface for a tool called LET. At the top, there is a dark header bar with the 'XNLP' logo on the left and 'Login Register' links on the right. The main content area has a 'Welcome Back' heading. Below this, there are two input fields: 'Email address' with a placeholder 'Enter email' and 'Password' with a placeholder 'Password'. A dark 'Sign In' button is positioned below the password field. At the bottom, there is a link that says 'Don't have an account? [Register here](#)'.

Login Screen

Figure 3: Login interface of LET.

7.3 Landing Page

The landing page serves as the entry point to LET, introducing users to the tool’s central purpose: enabling systematic exploration of explanation quality in large language models. The page highlights the key functionality—dataset classification, explanation generation, and evaluation of faithfulness and plausibility—while providing intuitive navigation. Two main pathways are offered: the **Dashboard**, which gives direct access to model-based classification and explanation analysis, and the **Datasets** view, which allows users to upload, manage, and evaluate larger collections of examples. The design is intentionally minimal, focusing the user’s attention on core tasks without unnecessary complexity.



Landing Page

Figure 4: Landing page of LET, where users can access either the Dashboard for direct classification or the Datasets view for dataset-level workflows.

7.4 Settings Page

The Settings page allows users to configure their environment by managing API keys and provider preferences. On the left-hand side, users can securely enter and update API keys for all supported providers (e.g., OpenAI, Groq, DeepSeek, Gemini, OpenRouter, and Ollama). At least one valid key must be provided to enable classification or explanation runs, though users are free to add multiple providers for flexibility.

On the right-hand side, provider preferences can be set independently for classification and explanation tasks. For example, a user might select OpenAI for classification while using Groq for explanation generation, enabling side-by-side comparisons across providers. These preferences are stored in the database and applied automatically during future runs, streamlining the workflow.

This design illustrates one of LET's central principles: **provider-agnostic modularity**. By separating classification and explanation settings, the tool allows heterogeneous workflows (e.g., using a smaller model for predictions but a larger model for explanations), reflecting both practical and experimental needs.

XNLP

My Datasets Logout

Settings

API Keys

Configure your API keys for different providers

OpenAI API Key

Enter your OpenAI API key

Grok API Key

Enter your Grok API key

DeepSeek API Key

Enter your Deepseek API key

Openrouter API Key

Enter your Openrouter API key

Gemini API Key

Enter your Gemini API key

Update API Keys

Classification Settings

Select the AI provider for classification tasks

Provider Selection

OpenAIGroqDeepseekOpenrouterGeminiOllama

Save Classification Preferences

Explanation Settings

Select the AI provider for explanation generation

Provider Selection

OpenAIGroqDeepseekOpenrouterGeminiOllama

Save Explanation Preferences

Settings Page

Figure 5: The Settings page in LET, where users configure API keys and set provider preferences for classification and explanation tasks.

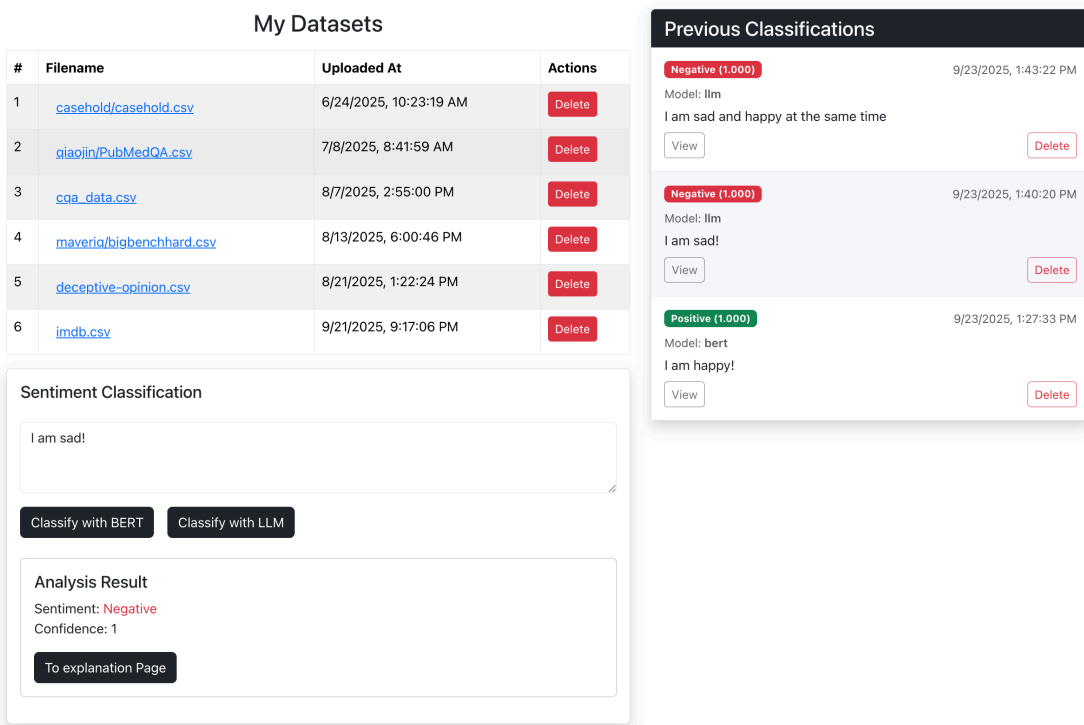
7.5 Dashboard Page

The Dashboard Page serves as the central access point for both dataset-level and single-sample interactions. On the left, the **My Datasets** panel lists all uploaded datasets with metadata (filename, upload timestamp) and basic actions such as deletion. Each dataset entry links to a dedicated dataset view for detailed classification and explanation runs. On the right, the **Previous Classifications** panel displays the outcomes of earlier single-text classifications, including the model used, confidence scores, timestamp, and options to revisit or delete results.

At the bottom of the interface, the **Sentiment Classification** box enables users to directly test the system on custom free-text inputs. Users can type a sentence (e.g., “I am sad!”), and classify it either with a transformer baseline (BERT) or with a connected large language model (LLM). The **Analysis Result** panel then shows the predicted sentiment, its confidence score, and provides a **“To Explanation Page”** button. Clicking this button transitions the user into the explanation workflow, where the system generates a natural-language explanation for the prediction. Explanations can be produced either directly by an LLM (self-explanation) or in combination with SHAP-based attributions (post-hoc augmentation). This feature closes the loop between classification and interpretability, allowing users to see not only the predicted label but also the underlying rationale.

Important Note: Free-text input is intentionally limited to **sentiment analysis**. This decision reflects the practical constraints of other supported tasks: legal reasoning

requires well-defined case holdings, biomedical QA requires domain-specific passages, sarcasm detection depends on carefully constructed ironic statements, and commonsense reasoning is only meaningful in structured multiple-choice settings. By contrast, sentiment analysis can be tested on everyday sentences provided by users, making it an intuitive “sandbox” task that introduces them to LET’s workflow. More complex tasks can still be explored, but only through dataset uploads.



Dashboard Page

Figure 6: Dashboard of LET, showing dataset management, previous classifications, and an interactive sentiment analysis module. The “To Explanation Page” button connects classification results to the explanation generation workflow.

7.6 Dataset Management Page

The Datasets page allows users to manage the data on which classification and explanation runs will be performed. Two upload modes are supported:

Hugging Face Import:

Users can directly pull from a set of supported datasets available on the Hugging Face Hub (e.g., IMDB, CaseHOLD, PubMedQA, SNARKS) by copying and pasting the dataset name from the Hugging Face website. This option ensures compatibility and standardized formatting for common benchmarks.

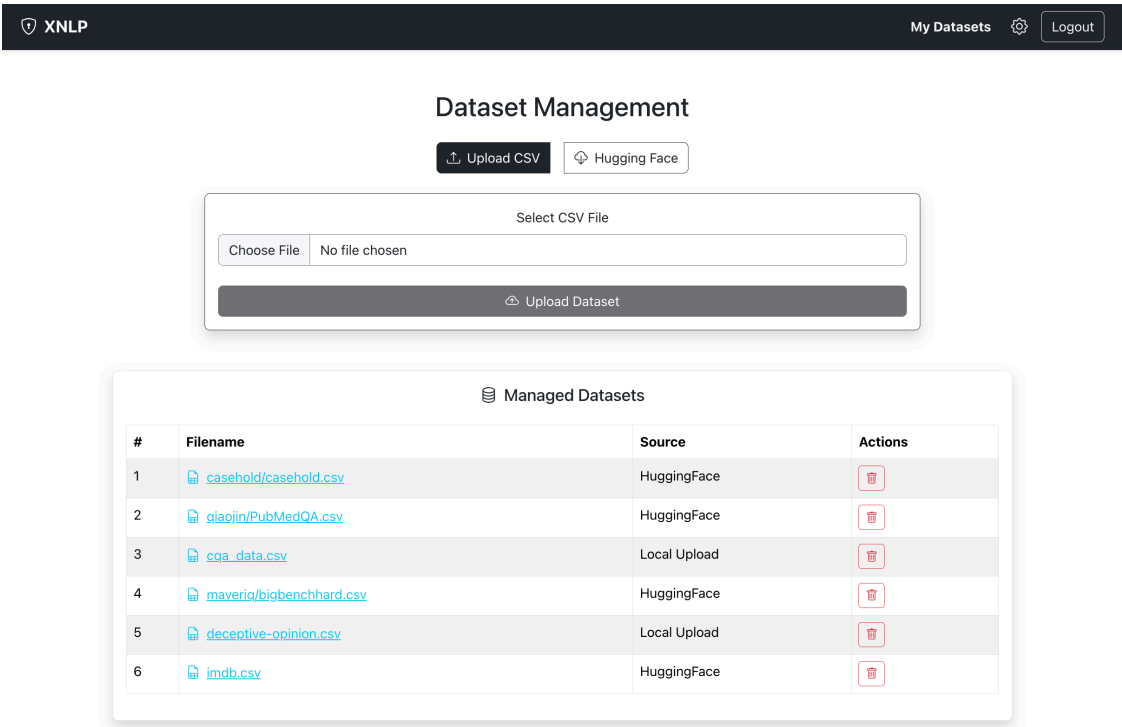
Local Upload:

Users can also upload their own data in CSV format. Even though LET is optimized for a

fixed set of datasets, this option extends flexibility, enabling users to experiment with personal or domain-specific datasets as long as they adhere to the expected schema (e.g., input text, label, choices).

Once uploaded, datasets are listed under **Managed Datasets** with metadata including filename, source (Hugging Face or local upload), and a delete option for removal. Links provided in the filename column allow users to directly open a dataset for classification and explanation runs.

Although the system is primarily designed around a limited set of benchmark datasets, this additional upload capability ensures extensibility and adaptability. Researchers may test LET on their own data collections, making the tool not only suitable for controlled experiments but also customizable for exploratory use cases in different domains.



Dataset Management Page

Figure 7: Dataset management page of LET. Users can upload datasets either from Hugging Face or via local CSV upload, and manage all available datasets in a central table.

7.7 Dataset View

The dataset view (Figure 8) shows the dataset-level workspace for running classifications and collecting explanations. The page is split into two functional regions:

Left panel — Classification Methods:

This panel contains all run controls:

1. **Back to Datasets:** Returns to the dataset manager
2. **Entries to Classify:** Specifies the batch size (how many rows from the current page to process in one run)
3. **Chain-of-Thought Prompting:** (toggle) Enables step-by-step reasoning in the prompt. This toggle is only available for datasets where CoT is relevant (e.g., ECQA, CaseHOLD, PubMedQA)
4. **Classify with LLM:** Performs label prediction only (no explanation). Useful for post-hoc workflows where explanations are requested later
5. **Classify and explain with LLM:** Triggers self-explanations: the model predicts and justifies its answer in a single prompt, enabling immediate faithfulness/plausibility evaluation
6. **Previous Classifications:** Lists recent runs for this dataset with the model/provider used and a confidence/score tag. Each card has a View action (to open the detailed result/explanation view) and Delete (to remove the stored run)

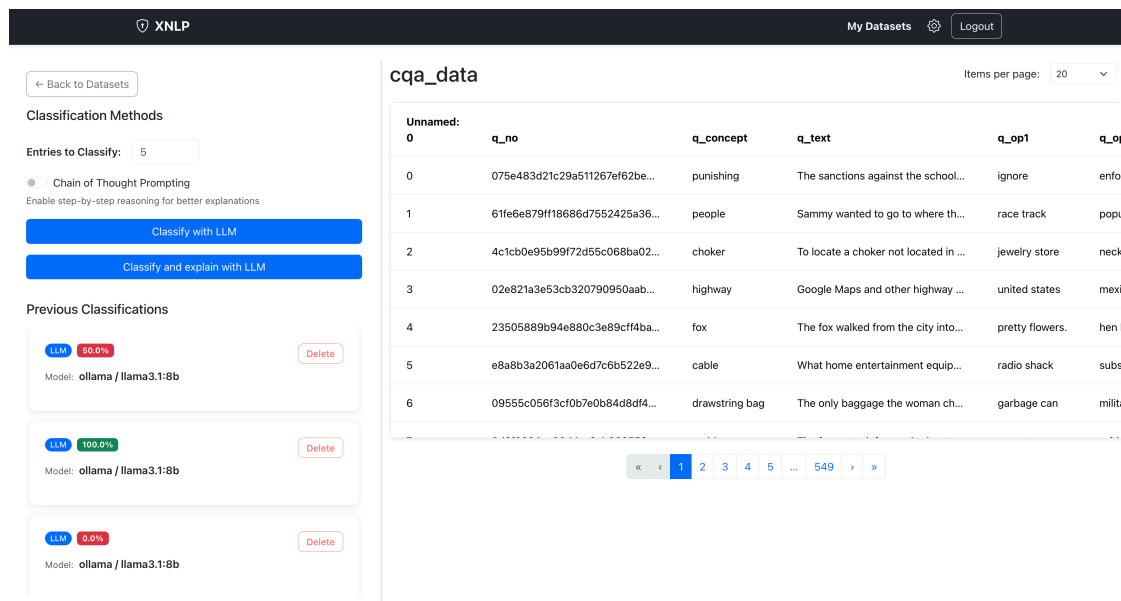
Right panel — Data table:

The main table renders the full dataset with all available columns, which vary depending on the dataset schema (e.g., for ECQA: question identifier, concept, question text, and answer options). Users can navigate the dataset using the “Items per page” selector and pagination controls at the bottom, which only affect the visible slice of data but do not alter stored runs.

Each row in the table is clickable: selecting an entry opens a preview window where the user can inspect the instance in more detail. From there, the “Go to Entry” action takes the user to a dedicated explanation view. In this view, the system retrieves the prediction and generates an explanation using the provider and model chosen in the Explanation Settings.

Workflow notes:

- Choosing **Classify and explain with LLM** stores both predictions and free-text rationales for the selected rows. In addition, LET automatically computes evaluation metrics at this stage: faithfulness metrics (QAG, Counterfactual Stability, Contextual Faithfulness) for all datasets, and plausibility metrics when gold-standard rationales are available (e.g., PubMedQA, ECQA). The results are combined into a trustworthiness score following the LExT framework. For datasets without gold explanations (e.g., IMDB, Deceptive Opinion, CaseHOLD, SNARKS), only faithfulness is computed.
- Selecting **Classify with LLM** yields labels only; explanations can be obtained later via a post-hoc prompt, potentially using a different model than the classifier (useful for isolating explanation quality from prediction quality).
- For the IMDB (sentiment) dataset, the page also includes a **Classify with BERT** button to run a local transformer baseline. In that view, confidence scores are shown and SHAP token importances can be requested.



Dataset View

Figure 8: Dataset view for ECQA. Left: run controls (batch size, Chain-of-Thought toggle, LLM classify vs. classify+explain, previous runs). Right: paginated table showing question fields and answer options.

7.8 Classification Dashboard

The Classification Dashboard (Figure 9) shows the summary report that opens after a dataset run completes. A compact header displays the run metadata as badges (Method=LLM/BERT, Provider=OpenAI/.../Ollama, Model=, Type=) and a “Back to datasetview” control. On the right, the “Choose Different LLMs” action lets users attach additional models for post-hoc explanations (e.g., explain an existing BERT prediction with a different LLM), enabling cross-model analysis without re-running the classification.

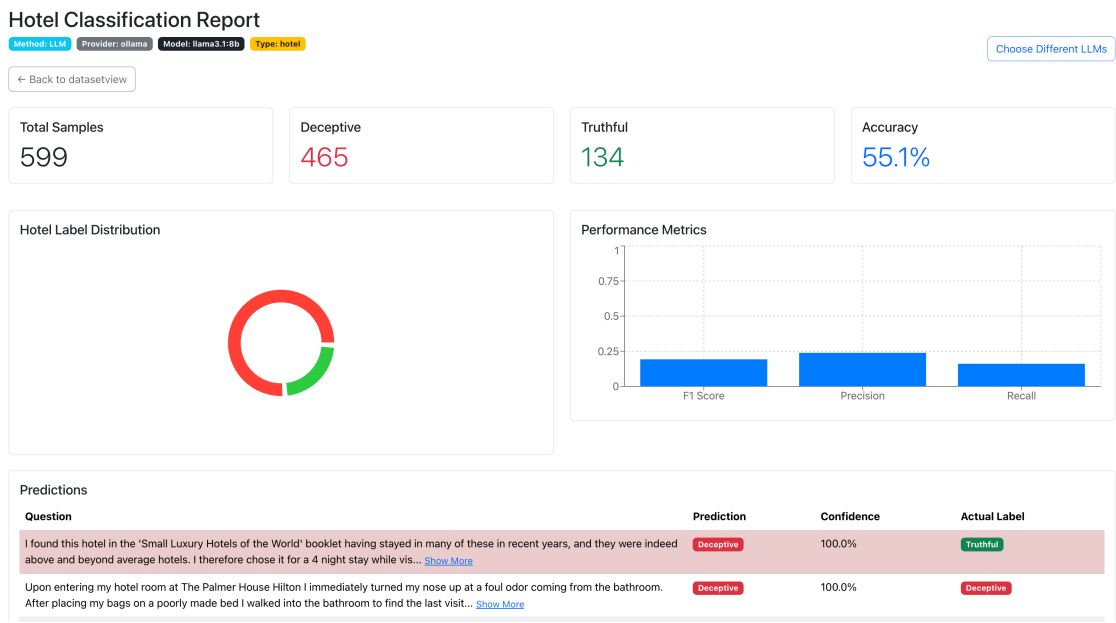
The top row of summary cards reports: 1. **Total Samples** processed in the run 2. The count per class (e.g., Deceptive, Truthful for the hotel-review dataset) 3. Overall **Accuracy**

Below, the left panel visualizes the **label distribution** as a donut chart so users can quickly gauge class imbalance. The right panel presents **Performance Metrics** (F1, Precision, Recall) as bars; these are computed whenever gold labels are available for the dataset.

The **Predictions** table lists each input (truncated with a “Show More” expander for long texts), the model’s **Prediction**, the reported or derived **Confidence** (actual confidence for BERT; set to 1.0 for LLMs), and the **Actual Label**. Rows are highlighted in red when the prediction disagrees with the ground truth, making failure cases immediately visible for follow-up analysis.

From here, users typically jump back to the dataset view (via the header control) to open an item’s Explanation page, where LEXt-style faithfulness metrics (QAG, Counterfactual

Stability, Contextual Faithfulness) and—when the dataset supplies gold rationales—plausibility metrics can be inspected in detail.



Classification Dashboard

Figure 9: Classification Dashboard view. The page summarizes dataset-level performance (samples, accuracy, class distribution, metrics) and lists individual predictions with confidence scores and ground-truth labels. Incorrect predictions are highlighted in red for quick error inspection.

7.9 Explanation Page (Instance View)

This view is designed for fine-grained, per-instance analysis and rating (Figure 10). It brings the original input, the model’s decision, and explanation-quality metrics into a single screen so users can inspect, regenerate, and rate explanations.

Header and navigation:

At the top-right, the pager (“Result i of N”) shows the current index within the run and offers Previous/Next buttons for rapid triage across items. A Back button returns to the corresponding classification summary.

Instance panel (input & labels):

The Question card displays the raw input text for the selected item. On the right, colored badges show the model’s Prediction and, when available, the Actual Label (ground truth). A confidence line appears beneath the input when the underlying model exposes a calibrated score.

Faithfulness and Plausibility metrics:

The Metrics card reports all available evaluation scores, based on the LExT framework. For every dataset, the three faithfulness metrics are shown:

1. **QAG** (Question-Answer Generation): Fraction of auxiliary questions—derived from the explanation—that can be answered from the explanation alone
2. **Counterfactual Stability**: Whether flipping the explanation to support an opposing label causes the model’s answer to flip accordingly (normalized to [0,1])
3. **Contextual Faithfulness**: The extent to which the model signals “insufficient information” after key tokens are redacted (higher is better)

In addition, for datasets that provide gold rationales (e.g., PubMedQA, ECQA), plausibility metrics are also displayed:

1. **Correctness**: Semantic similarity between the generated explanation and the ground-truth rationale, measured via BERT embeddings (with optional NER-weighting for biomedical texts)
2. **Consistency**: Stability of explanations under repeated queries or paraphrased inputs, capturing robustness of generated rationales

This design ensures that when gold explanations are available, both plausibility and faithfulness can be evaluated side by side. In datasets without ground-truth rationales, only faithfulness metrics are shown.

LLM explanations (generation & rating):

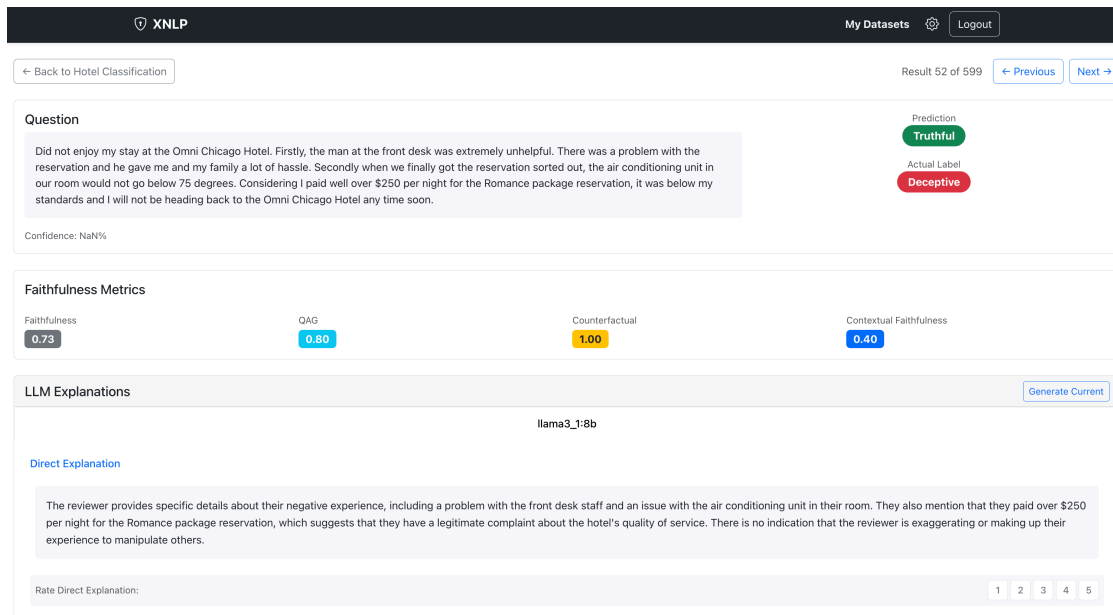
The LLM Explanations section shows the active explainer model (e.g., llama3_1:8b) and presents the current Direct Explanation (self-explanation). The “Generate Current” button regenerates the explanation using the provider/model configured in Settings; doing so updates the explanation and re-computes faithfulness metrics for this instance. A 1-5 rating widget (Rate Direct Explanation) lets users provide a plausibility/quality judgment; ratings are persisted to the database for later analysis.

Dataset-specific behavior:

For datasets with human rationales, the page can additionally surface plausibility measurements (e.g., semantic similarity to references) alongside faithfulness. For datasets without references (e.g., Deceptive Opinion), only faithfulness metrics and the human rating UI are shown.

Workflow fit:

Typical use is: navigate to an error or borderline case, regenerate an explanation with the selected provider, inspect the three faithfulness dimensions, optionally provide a human rating, and move to the Next item. This complements the dataset-level reports by enabling targeted, item-level diagnosis.



Explanation Page

Figure 10: Explanation page (instance view). The top panel displays the original input and model prediction alongside the ground-truth label. Faithfulness metrics (QAG, Counterfactual, Contextual) are automatically computed and reported for the selected explanation. The lower section shows the generated LLM explanation, which can be regenerated (Generate Current) and rated on a 1-5 scale by the user. Navigation controls allow moving between dataset entries, enabling fine-grained inspection and rating of explanations.

7.10 Explanation Page with SHAP Integration

Figure 11 illustrates the explanation interface when SHAP-based feature attribution is enabled for sentiment analysis. This page combines local feature importance visualizations with LLM-generated textual rationales to provide a multi-perspective view of model reasoning.

At the top, the **Original Text** panel shows the review or input instance in full. On the right-hand side, the predicted label (e.g., Negative) and the ground-truth label are displayed. Confidence scores are shown when available, making the system’s uncertainty explicit. Incorrect predictions are highlighted in red to draw attention to potential model failures.

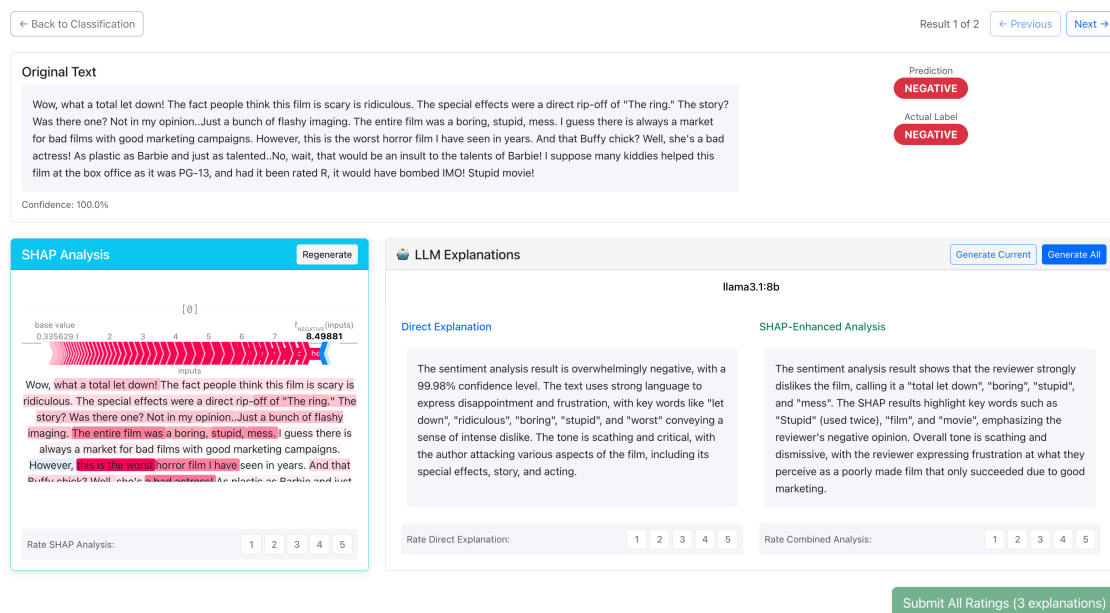
The left panel presents the **SHAP Analysis**. Here, the review text is augmented with token-level color overlays indicating the contribution of each word to the model’s decision. Words highlighted in darker shades (e.g., “stupid”, “boring”, “worst”) are the strongest contributors toward the negative classification. Users can regenerate the SHAP analysis if they wish to update or confirm the attribution for the same instance. Below the visualization, a 1-5 rating scale allows users to judge the usefulness of the SHAP explanation, providing human-centered feedback on interpretability.

On the right, the **LLM Explanations** panel provides textual rationales. Two modes are offered:

- **Direct Explanation:** The LLM is prompted to explain the prediction in natural language without external input, highlighting key phrases and providing an overall interpretation of the review sentiment.
- **SHAP-Enhanced Analysis:** The LLM receives the SHAP-highlighted words and incorporates them into the explanation. This results in a hybrid approach where statistical feature attribution (SHAP) is combined with human-readable narrative reasoning from the LLM.

Each explanation can be rated individually by users using a 1-5 scale, enabling the collection of plausibility judgments. A submission button (Submit All Ratings) ensures that feedback for multiple explanations is stored consistently.

This dual presentation of explanations is designed to bridge two complementary interpretability paradigms: feature-based attribution (SHAP) and free-text rationales (LLMs). Together, they allow researchers to evaluate whether explanations are both faithful to model internals and plausible to human users.



Explanation Page with SHAP

Figure 11: Explanation page with SHAP integration. The interface combines feature attribution (left) with natural language explanations (right), enabling comparison between purely model-derived importance scores and narrative rationales. Incorrect predictions are highlighted, and users can rate each explanation's quality.

7.11 Workflow Summary

Together, these interfaces form the complete workflow of LET:

1. **Register/Login** → Configure API keys and provider preferences
2. **Upload/Select Dataset** → Choose from benchmarks or upload custom data
3. **Dataset View** → Configure batch size, CoT, and classification mode
4. **Run Classification** → Generate predictions (with or without explanations)
5. **Classification Dashboard** → Review aggregate performance and identify errors
6. **Explanation Page** → Inspect individual instances, view metrics, regenerate explanations
7. **Rate Explanations** → Provide human feedback on explanation quality
8. **Export Results** → Download predictions, explanations, and metrics for analysis

By combining cloud-based LLMs, locally deployable models, and feature-attribution methods such as SHAP, the system provides a unified platform for exploring the faithfulness and plausibility of AI explanations. Researchers can upload their own datasets or use supported benchmarks, select providers and explanation modes, and immediately inspect results through interactive dashboards. Importantly, LET not only generates explanations but also computes standardized evaluation metrics, enabling systematic comparison across tasks, models, and explanation strategies. In this way, the software bridges practical usability with experimental rigor, making it both a research tool for controlled studies and a practical environment for applied explainable NLP.

8. Installation and Setup

LET is designed to be deployed either locally (for development and private use) or on a web server (for multi-user access). This section provides comprehensive installation instructions for both scenarios.

8.1 Prerequisites

Before installing LET, ensure that the following software is installed on your system:

- **Python 3.10 or higher** (3.10+ recommended for compatibility with modern ML libraries)
- **Node.js 16 or higher** (for React frontend build tools)
- **MongoDB 4.4 or higher** (for database storage)
- **Git** (for cloning the repository)

Optional: - **Ollama** (for local LLM deployment - only needed if you plan to run models locally) - **GPU with CUDA support** (recommended for BERT classification and SHAP computation)

8.2 Docker Installation (Recommended)

The easiest way to run LET is using Docker Compose. This method automatically sets up MongoDB, backend, and frontend with a single command.

Prerequisites

- [Docker Desktop](#) installed and running

Quick Start

1. **Clone the repository** (if you haven't already):

```
git clone <your-repo-url>
cd thesisXNLP
```

2. **Start all services:**

```
docker compose watch
```

Or for standard mode:

```
docker compose up --build
```

3. **Access the application:**

- Frontend: `http://localhost:3000`
- Backend API: `http://localhost:5000`
- MongoDB: `localhost:27017`

4. **Stop services:**

```
docker compose down
```

Note: Modern Docker uses `docker compose` (without hyphen). If that doesn't work, use `docker-compose` (with hyphen) instead.

What Docker Sets Up

- **MongoDB:** Automatically started in a container with persistent data storage
- **Backend:** Flask application with all dependencies installed
- **Frontend:** React development server with hot reload
- **Public Datasets:** Automatically seeded on backend startup (casehold.csv, imdb.csv, etc.)

Development Mode with Watch

Using `docker compose watch` provides: -  Automatic file syncing (no rebuild needed for code changes) -  Flask auto-reload on Python file changes -  React hot-reload on frontend changes -  Automatic rebuilds when dependencies change

Viewing Logs

View all logs

```
docker compose logs -f
```

View backend Logs (Flask HTTP requests)

```
docker logs -f backend
```

View frontend Logs

```
docker logs -f frontend
```

View logs with timestamps

```
docker compose logs -f --timestamps backend
```

View last 100 lines of backend logs

```
docker compose logs --tail=100 backend
```

Environment Variables (Optional)

All environment variables have defaults, so `.env` file is optional. Create one only to customize values.

Development defaults: - FLASK_SECRET_KEY: your-secret-key-change-in-production -
REACT_APP_API_URL: http://localhost:5000 - FRONTEND_ORIGIN:
http://localhost:3000

Create a `.env` file in the project root to customize settings:

```
FLASK_SECRET_KEY=your-secure-secret-key-here  
REACT_APP_API_URL=http://localhost:5000  
FRONTEND_ORIGIN=http://localhost:3000
```

Useful Development Commands

Start with watch mode (recommended for development)

```
docker compose watch
```

Standard start

```
docker compose up --build
```

View Logs

```
docker compose logs -f backend
```

Stop services

```
docker compose down
```

Execute command in container

```
docker compose exec backend python --version
```

```
docker compose exec mongodb mongosh
```

Rebuild specific service


```
docker compose build backend
```

```
# Stop and remove containers and volumes
```

```
docker compose down -v
```

Development Services Overview

MongoDB: - Container name: mongodb - Port: 27017 - Data persistence: Named volume mongodb_data - Database: auth_app

Backend (Flask): - Container name: backend - Port: 5000 - Environment variables: - MONGO_URI: MongoDB connection string (set automatically) - FLASK_SECRET_KEY: Secret key for Flask sessions - FLASK_DEBUG: Debug mode (default: 1) - FRONTEND_ORIGIN: Frontend origin for CORS (default: http://localhost:3000) - Volumes: Source code mounted for live editing (./backend:/app)

Frontend (React): - Container name: frontend - Port: 3000 - Environment variables: - REACT_APP_API_URL: Backend API URL (default: http://localhost:5000) - CHOKIDAR_USEPOLLING: Enable file watching in Docker (set to true) - Volumes: Source code mounted for hot reload (./explainable-nlp/src:/app/src)

Production Mode

For production deployment, use the production Docker Compose file (docker-compose.prod.yml):

Build and Start:

```
# Build production images
```

```
docker compose -f docker-compose.prod.yml build
```

```
# Start production services
```

```
docker compose -f docker-compose.prod.yml up -d
```

```
# OR rebuild and restart in one command
```

```
docker compose -f docker-compose.prod.yml up -d --build
```

Access: - Frontend: http://localhost:80 (or http://localhost) - Backend API: http://localhost:5000 (direct) or via frontend at http://localhost/api - MongoDB: localhost:27017

Stop Production Services:

```
docker compose -f docker-compose.prod.yml down
```

Production Features

The production setup (docker-compose.prod.yml) is optimized for production deployment:

- **Multi-stage Docker builds:** Smaller final images by excluding build dependencies

- **Pre-downloaded ML models:** Models (BERT, DistilBERT) are downloaded during Docker build, not at runtime
- **Lazy model loading:** Models load only when first used, not at startup
- **Gunicorn WSGI server:** Production-grade WSGI server instead of Flask dev server
- **Nginx:** Serving React build and proxying API requests
- **Resource limits:** CPU and memory limits for all services
- **Health checks:** Automatic service monitoring and dependency management
- **Code baked into images:** No source code volumes - code is baked into images for better performance

Production Services Overview

MongoDB (Production): - Container name: mongodb-prod - Port: 27017 - Data persistence: Named volume mongodb_data - Resource limits: 1 CPU, 2GB RAM - Health check: MongoDB ping command

Backend (Production): - Container name: backend-prod - Port: 5000 - WSGI Server: Gunicorn (2 workers, 2 threads) - Environment variables: - FLASK_DEBUG=0 (production mode) - FLASK_ENV=production - FRONTEND_ORIGIN=http://localhost:80 - SECRET_KEY: Required for Fernet encryption - Resource limits: 2 CPUs, 4GB RAM - Health check: HTTP endpoint /api/health - Volumes: Only uploads directory (not source code)

Frontend (Production): - Container name: frontend-prod - Port: 80 - Web Server: Nginx - Build: React production build served as static files - API Proxy: Nginx proxies /api/* requests to backend - Resource limits: 1 CPU, 1GB RAM - Health check: HTTP endpoint check

Production Environment Variables

All environment variables have defaults, so .env file is optional. Create one only to customize values.

Production defaults: - FLASK_SECRET_KEY: your-secret-key-change-in-production - SECRET_KEY: Same as FLASK_SECRET_KEY - FRONTEND_ORIGIN: http://localhost:80 - REACT_APP_API_URL: /api

Note: For production, it's recommended to set strong secret keys in .env file for security.

Production Volumes

- **MongoDB data:** Persisted in Docker volume mongodb_data
- **Backend uploads:** Mounted from ./backend/uploads (data only)
- **Transformers cache:** Optional volume transformers_cache for model persistence

Note: Source code is NOT mounted in production - it's baked into Docker images.

Useful Production Commands

Build images

```
docker compose -f docker-compose.prod.yml build
```

Start services

```
docker compose -f docker-compose.prod.yml up -d
```

View Logs

```
docker compose -f docker-compose.prod.yml logs -f
```

View backend Logs

```
docker compose -f docker-compose.prod.yml logs -f backend
```

View frontend Logs

```
docker compose -f docker-compose.prod.yml logs -f frontend
```

Stop services

```
docker compose -f docker-compose.prod.yml down
```

Rebuild and restart

```
docker compose -f docker-compose.prod.yml up -d --build
```

Check service status

```
docker compose -f docker-compose.prod.yml ps
```

View resource usage

```
docker stats
```

Docker Troubleshooting

MongoDB connection issues:

Check service status

```
docker compose ps
```

OR for production

```
docker compose -f docker-compose.prod.yml ps
```

View MongoDB Logs

```
docker compose logs mongodb
```

OR for production

```
docker compose -f docker-compose.prod.yml logs mongodb-prod
```

Watch Configuration

The docker-compose.yml includes watch configuration that:

- **Backend:** Rebuilds when requirements.txt changes
- **Frontend:** Rebuilds when package.json changes

Note: File syncing is handled automatically by bind mount volumes (./backend:/app and ./explainable-nlp/src:/app/src), so we only use watch for rebuilds when dependencies change. This eliminates warnings about duplicate path monitoring.

Watch actions: - rebuild: Rebuilds the container image when dependency files change (requirements.txt, package.json)

8.2 Repository Structure

```
thesisXNLP/
├── backend/                                # Python Flask backend
│   ├── app.py                            # Main Flask application
│   ├── routes/                          # API endpoints
│   ├── LExT/                            # Evaluation framework implementation
│   │   ├── src/                         # Core LExT components
│   │   └── metrics/                    # Faithfulness and plausibility metrics
│   ├── requirements.txt                  # Python dependencies
│   └── .env.example                     # Environment variable template
├── explainable-nlp/                      # React TypeScript frontend
│   ├── src/
│   │   ├── components/                 # UI components
│   │   └── index.tsx                  # Entry point
│   ├── package.json                    # Node.js dependencies
│   └── tsconfig.json                  # TypeScript configuration
└── README.md                           # Project overview
```

8.3 Manual Installation (Alternative)

If you prefer to run services manually without Docker, follow the steps below.

8.4 Backend Setup

Step 1: Clone the Repository

```
git clone https://github.com/yarkinerenn/LET.git
cd thesisXNLP
```

Step 2: Install Conda

If you don't have Conda installed:

macOS / Linux:

```
# Download Miniconda (recommended) or Anaconda
# Miniconda: https://docs.conda.io/en/latest/miniconda.html
# Anaconda: https://www.anaconda.com/products/distribution
```

```
# After installation, restart your terminal or run:
source ~/.bashrc # or source ~/.zshrc for zsh
```

Windows: - Download and install Miniconda or Anaconda from: - Miniconda:
<https://docs.conda.io/en/latest/miniconda.html> - Anaconda:
<https://www.anaconda.com/products/distribution> - Use Anaconda Prompt for terminal commands

Step 3: Create Environment and Install Dependencies

```
conda env create -f environment.yml          # Run once to create the environment
conda activate let                           # Run in every new shell before using the backend
```

Key Dependencies: - Flask: Web framework - PyTorch: Deep learning backend - Transformers: Hugging Face model library - SHAP: Feature attribution - pymongo: MongoDB driver - python-dotenv: Environment variable management - requests: HTTP client for provider APIs

Step 4: Configure Environment Variables

Create a .env file in the backend/ directory:

```
cd backend
cp .env.example .env                      # On Windows use: copy .env.example .env
# Edit .env with your MongoDB URI and other settings
```

Edit .env with your configuration:

```
FLASK_SECRET_KEY=change-me
MONGO_URI=mongodb://localhost:27017/auth_app      # Local development
SESSION_COOKIE_NAME=let_session
UPLOAD_FOLDER=uploads
```

Flask initializes the MongoDB client through `mongo.init_app(app)` on startup, so as long as the URI is reachable the database and collections will be created automatically on first write. Make sure the MongoDB service is running before launching Flask.

8.5 MongoDB Setup

Local MongoDB (development)

macOS 1. Install MongoDB Community Edition:

```
bash brew tap mongodb/brew brew install mongodb-community@7.0 2. Start the service:
```

```
bash brew services start mongodb/brew/mongodb-community 3. Verify it is running:
bash mongosh
```

Ubuntu / Debian 1. Import the MongoDB public key and add the repository (example for 7.0):

```
bash curl -fsSL https://pgp.mongodb.com/server-7.0.asc | \ sudo gpg -
o /usr/share/keyrings/mongodb-server-7.0.gpg --dearmor echo "deb [signed-
```

```
by=/usr/share/keyrings/mongodb-server-7.0.gpg] \
https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0 multiverse" | \
sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list    sudo apt-get update
sudo apt-get install -y mongodb-org 2. Start and enable the service:
bash    sudo systemctl start mongod    sudo systemctl enable mongod 3. Check
status:
bash    sudo systemctl status mongod
```

Windows 1. Download the MSI installer from <https://www.mongodb.com/try/download/community> (choose the latest stable version). 2. Run the installer and select **Install MongoDB as a Service** (default settings are fine). 3. After installation, open **Command Prompt** and run:
cmd mongosh If it opens the shell, the server is running. If not, start the service via **Services** → **MongoDB Server** → **Start**.

Optional seeding (all platforms) - Import seed data with mongorestore --uri "<MONGO_URI>" dump/ or execute scripts via mongosh.

Step 5: Run Flask Backend

```
cd backend
python app.py
```

The backend should now be running on <http://localhost:5000>.

Expected Output:

```
* Running on http://127.0.0.1:5000
* Debug mode: on
```

8.6 Frontend Setup

Step 1: Navigate to Frontend Directory

Open a new terminal window (make sure Conda environment is active):

```
conda activate let
cd thesisXNLP/explainable-nlp
```

Step 2: Install Node Dependencies

```
npm install
```

Key Dependencies: - React: UI framework - TypeScript: Type safety - Axios: HTTP client - Chart.js: Visualization library - React Router: Navigation

Step 3: Configure Frontend Environment

Create .env file in explainable-nlp/ directory (optional):

```
REACT_APP_API_URL=http://localhost:5000
```

Step 4: Start Development Server

```
npm start
```

The frontend should automatically open in your browser at <http://localhost:3000>.

Expected Output:

Compiled successfully!

You can now view explainable-nlp in the browser.

```
Local:          http://localhost:3000
On Your Network: http://192.168.1.x:3000
```

8.7 API Key Configuration

After installation, you must configure at least one provider API key to use LET. There are two ways to do this:

Option 1: During Registration

When creating a new account, the registration screen includes optional fields for API keys. Enter keys for the providers you wish to use.

Option 2: In Settings After Login

1. Log in to LET
2. Navigate to Settings (gear icon or Settings menu)
3. Enter API keys in the Provider Configuration section
4. Click Save

Supported Providers: - OpenAI: <https://platform.openai.com/api-keys> - Gemini: <https://makersuite.google.com/app/apikey> - DeepSeek: <https://platform.deepseek.com/> - Groq: <https://console.groq.com/keys> - OpenRouter: <https://openrouter.ai/keys>

8.8 Local Model Support (Ollama)

To enable local LLM deployment with Ollama:

Step 1: Install Ollama

Visit <https://ollama.ai> and follow installation instructions for your operating system.

```
# macOS
```

```
brew install ollama
```

```
# Linux
```

```
curl -fsSL https://ollama.ai/install.sh | sh
```

Windows
Download installer from <https://ollama.ai/download>

Step 2: Pull a Model

```
ollama pull llama2           # Llama 2 (7B parameters)
ollama pull llama3.1:8b      # Llama 3.1 (8B parameters)
ollama pull mistral          # Mistral 7B
```

Step 3: Verify Ollama is Running

```
ollama list
# Should show downloaded models
```

```
curl http://localhost:11434/api/tags
# Should return JSON with available models
```

Step 4: Configure LET to Use Ollama

In the LET Settings page, select “Ollama” as the provider and choose your downloaded model from the dropdown.

Important Notes: - Ollama requires significant local resources (GPU with 8GB+ VRAM recommended) - Inference speed depends on hardware; expect slower responses compared to cloud APIs - Ollama is only available in local deployments of LET

8.9 Running the App

Using Docker (Recommended)

If you installed using Docker, simply run:

```
docker compose watch
```

This starts all services (MongoDB, backend, frontend) automatically. Access the app at <http://localhost:3000>.

Manual Setup (Alternative)

If you installed manually, make sure the Conda environment is active (`conda activate let`) and run the appropriate script from the project root:

- **macOS / Linux**

```
./start.sh
```

- **Windows (Anaconda Prompt / PowerShell)**

```
start_windows.bat
```

The script opens a new terminal window for the frontend (`npm start`) and keeps the backend (`python app.py`) in the current window. Stop the backend with `Ctrl+C` and close the frontend window when you are done.

8.5 API Key Configuration

After installation, you must configure at least one provider API key to use LET. There are two ways to do this:

Option 1: During Registration

When creating a new account, the registration screen includes optional fields for API keys. Enter keys for the providers you wish to use.

Option 2: In Settings After Login

1. Log in to LET
2. Navigate to Settings (gear icon or Settings menu)
3. Enter API keys in the Provider Configuration section
4. Click Save

Supported Providers: - OpenAI: <https://platform.openai.com/api-keys> - Gemini: <https://makersuite.google.com/app/apikey> - DeepSeek: <https://platform.deepseek.com/> - Groq: <https://console.groq.com/keys> - OpenRouter: <https://openrouter.ai/keys>

8.6 Local Model Support (Ollama)

To enable local LLM deployment with Ollama:

Step 1: Install Ollama

Visit <https://ollama.ai> and follow installation instructions for your operating system.

macOS

```
brew install ollama
```

Linux

```
curl -fsSL https://ollama.ai/install.sh | sh
```

Windows

Download installer from <https://ollama.ai/download>

Step 2: Pull a Model

```
ollama pull llama2           # Llama 2 (7B parameters)
ollama pull llama3.1:8b      # Llama 3.1 (8B parameters)
ollama pull mistral           # Mistral 7B
```

Step 3: Verify Ollama is Running

```
ollama list
```

Should show downloaded models

```
curl http://localhost:11434/api/tags
```

Should return JSON with available models

Step 4: Configure LET to Use Ollama

In the LET Settings page, select “Ollama” as the provider and choose your downloaded model from the dropdown.

Important Notes: - Ollama requires significant local resources (GPU with 8GB+ VRAM recommended) - Inference speed depends on hardware; expect slower responses compared to cloud APIs - Ollama is only available in local deployments of LET

9. Experimental Analysis and Results Processing

While LET provides the infrastructure for generating explanations and collecting user data, a separate analysis pipeline processes experimental results from user studies conducted for this master thesis. This section documents the **result scripts** directory, which contains Python scripts for statistical analysis of user study data collected via Google Forms. The scripts take raw experimental data in Excel format (xlsx) exported from Google Forms, transform and process it, and then perform comprehensive statistical, exploratory, and descriptive analysis. These scripts are designed to analyze data collected from experiments conducted using LET, particularly focusing on how explanation faithfulness and model size affects user reliance, accuracy, and confidence.

9.1 Overview and Purpose

The result scripts provide a comprehensive analysis pipeline for processing experimental data from user studies conducted with LET as part of this master thesis research. The primary user study used the **Deceptive Opinion** dataset (see Section 4.2) to investigate how varying levels of explanation faithfulness affect human-AI interaction.

Data Collection and Processing Workflow: 1. **Raw Data Collection:** Experimental data is collected via Google Forms during user studies 2. **Data Export:** Raw data is exported from Google Forms in Excel format (xlsx files) 3. **Data Transformation:** The scripts transform raw Google Forms data into structured formats suitable for analysis 4. **Statistical Analysis:** Comprehensive statistical analysis including hypothesis testing, exploratory analysis, and descriptive statistics

The analysis pipeline processes data from multiple sources:

- **Prolific participants:** Two forms (A and B) with different orderings of model sizes
- **Friends and family participants:** Two forms matching the Prolific structure

The experimental design follows a **within-subjects (repeated measures)** approach, where each participant encounters both faithful and unfaithful explanations across different trials. This design requires specialized statistical methods that account for within-participant correlation, implemented using cluster-robust standard errors.

Key Capabilities: - **Data Transformation:** Converting raw Google Forms xlsx exports into structured datasets - **Descriptive Analysis:** Summary statistics, distributions, and exploratory data analysis - **Statistical Analysis:** Hypothesis testing with cluster-robust standard errors - **Metric Computation:** Calculation of behavioral metrics (RAIR, RSR, accuracy, confidence deltas) - **Demographic Subgroup Analysis:** Examining how demographic factors moderate effects - **Visualization Generation:** Creating plots and figures for results presentation - **Export:** Generating summary tables and statistics for thesis reporting

9.2 Directory Structure

The analysis pipeline is organized in the `result scripts/` directory, which follows a modular structure for maintainability:

```
result scripts/
├── main.py                # Main entry point - runs all analyses
├── config.py              # Centralized configuration (paths, settings)
├── requirements.txt       # Python dependencies
├── README.md              # Detailed usage documentation
├── scripts/
│   ├── data_processing/
│   │   └── process_data.py # Processes raw Prolific data, computes metrics
│   ├── analysis/
│   │   ├── hypothesis_tests.py # Main hypothesis testing (16 hypotheses)
│   │   └── demographics_table.py # Generates LaTeX demographics table
│   ├── demographic_analysis/
│   │   ├── age_analysis.py
│   │   ├── gender_analysis.py
│   │   ├── cs_expertise_analysis.py
│   │   ├── education_level_analysis.py
│   │   └── nlp_experience_analysis.py
│   └── utils/
│       └── plots.py        # Plotting utilities (shared functions)
├── data/                  # Input/output data files
├── plots/                 # Generated plots
│   ├── general/           # Main analysis plots
│   └── demographic_analyses/ # Demographic-specific plots
```

Main Entry Point: `main.py` provides a command-line interface to run all analyses or specific components. It handles dependency checking and orchestrates the execution of individual analysis scripts.

Configuration: `config.py` centralizes all paths, experiment settings, and constants. Paths are resolved relative to the config file location, making scripts robust to where they are executed from. Key configuration includes: - Data file paths (four input files) - Experiment parameters (number of trials, ground truth labels, AI predictions, faithfulness labels) - Output directories for plots and CSV files - Demographic column mappings - Statistical significance levels

9.3 Data Processing Pipeline

The data processing stage (`scripts/data_processing/process_data.py`) transforms raw experimental data exported from Google Forms and computes key behavioral metrics. The script takes Excel files (xlsx format) that were exported from Google Forms responses and processes them into a structured dataset suitable for statistical analysis.

Input Files

The pipeline processes four Excel files (exported from Google Forms) located in the `data/` directory:

File	Source	Form	Description
Prolific-big-small.xlsx	Prolific participants	Form A	Big LLM first, small LLM second
Prolific-small-big.xlsx	Prolific participants	Form B	Small LLM first, big LLM second
experiment.xlsx	Friends/family	Form A	Big LLM first, small LLM second
experiment2.xlsx	Friends/family	Form B	Small LLM first, big LLM second

All four files are automatically combined during processing, with form assignments tracked to account for order effects.

Computed Metrics

The data processing script computes several key metrics that measure user reliance and decision-making:

RAIR (Reliance After Initial Response): - Measures the proportion of cases where users correct their initial wrong answer after receiving AI advice - Only computed when: AI is correct AND human initially wrong - Formula: $RAIR = \text{corrected} / (\text{corrected} + \text{not_corrected})$ - Higher RAIR indicates greater reliance on correct AI advice

RSR (Reliance Shift Rate): - Measures the proportion of cases where users maintain their correct answer despite wrong AI advice - Only computed when: Human initially correct AND AI wrong - Formula: $RSR = \text{stayed_correct} / (\text{stayed_correct} + \text{switched_to_wrong})$ - Higher RSR indicates resistance to incorrect AI advice (appropriate skepticism)

Accuracy Metrics: - **Initial accuracy:** User accuracy before seeing AI explanations - **Final accuracy:** User accuracy after seeing AI explanations - Computed per trial and aggregated across participants

Confidence Deltas: - Change in user confidence ratings between initial and final responses - Positive deltas indicate increased confidence after seeing explanations - Negative deltas indicate decreased confidence

Output: The processed data is saved to `data/experiment_results_with_metrics.xlsx`, which serves as the input for all subsequent hypothesis testing and demographic analyses.

9.4 Hypothesis Testing Framework

The hypothesis testing module (`scripts/analysis/hypothesis_tests.py`) implements rigorous statistical analysis for the within-subjects experimental design. The script tests **16 hypotheses** related to how explanation faithfulness and model size affect user behavior.

Statistical Methodology

Cluster-Robust Standard Errors: - All regression models use cluster-robust standard errors, clustering by participant - This accounts for within-participant correlation in repeated measures designs - Prevents underestimation of standard errors that would occur with standard regression - Implemented using `statsmodels` with `cov_type='cluster'`

Model Selection: - **Binary outcomes** (RAIR, RSR, accuracy): Logistic regression with cluster-robust SEs - **Continuous outcomes** (confidence, plausibility): Ordinary least squares (OLS) with cluster-robust SEs - All models include appropriate control variables and interaction terms

Within-Subjects Comparisons: - Participant-level paired comparisons for faithful vs. unfaithful conditions - Normality tests (Shapiro-Wilk) on differences - If normal: Paired t-test with Cohen's d (paired) effect size - If non-normal: Wilcoxon signed-rank test with rank-biserial correlation

Significance Levels: - * $p < 0.05$ - ** $p < 0.01$ - *** $p < 0.001$

Hypothesis Categories

The 16 hypotheses cover multiple dimensions:

1. **Faithfulness effects** on reliance (RAIR, RSR)
2. **Faithfulness effects** on accuracy (initial, final, change)
3. **Faithfulness effects** on confidence changes
4. **Model size effects** on reliance and accuracy
5. **Interaction effects** between faithfulness and model size
6. **Plausibility ratings** and their relationship to reliance
7. **Agreement effects** (when human and AI agree/disagree)

Outputs

The hypothesis testing script generates:

- **CSV summary tables:**
 - `hypothesis_summary_table.csv`: Complete results for all 16 hypotheses
 - `participant_level_aggregates.csv`: Participant-level summary statistics
 - `within_subjects_comparisons_faithfulness.csv`: Paired comparisons for faithfulness conditions
 - `within_subjects_comparisons_modelsize.csv`: Paired comparisons for model size conditions
- **24 visualization plots** (saved to `plots/general/`):
 - Mean RAIR/RSR by faithfulness and model size
 - Accuracy distributions and per-question breakdowns
 - Confidence change analyses
 - Plausibility ratings and relationships
 - Scatter plots for agreement and reliance metrics

9.5 Demographic Analysis

Separate analysis scripts examine how demographic factors moderate the effects of explanation faithfulness. Each demographic analysis (`scripts/demographic_analysis/*.py`) follows a consistent structure:

Available Analyses: - `age_analysis.py`: Age group effects - `gender_analysis.py`: Gender differences - `cs_expertise_analysis.py`: Computer Science/AI expertise levels - `education_level_analysis.py`: Education level effects - `nlp_experience_analysis.py`: NLP experience effects

Statistical Approach: - Each analysis uses cluster-robust regression with demographic variables as moderators - Tests interactions between demographic factors and faithfulness/model size - Generates subgroup-specific effect estimates

Visualizations: - Box plots and violin plots showing distributions by demographic groups - Scatter plots with regression lines for continuous demographics - Grouped comparisons highlighting differences across subgroups

Demographics Table: The `demographics_table.py` script generates a LaTeX-formatted table summarizing participant demographics, suitable for inclusion in research papers. It computes counts, percentages, and summary statistics for all demographic variables.

9.6 Usage and Workflow

The analysis pipeline is executed via the command-line interface provided by `main.py`. The script supports running all analyses or specific components:

Run All Analyses (Recommended):

```
cd "result scripts"
python main.py
```

This executes the complete pipeline: 1. Data processing (combines files, computes metrics) 2. Hypothesis tests (16 hypotheses with cluster-robust SEs) 3. Demographics table generation 4. All demographic analyses

Run Specific Components:

Data processing only

```
python main.py --data-only
```

Hypothesis tests only

```
python main.py --hypothesis-only
```

ALL demographic analyses

```
python main.py --demographics-only
```

Specific demographic analysis

```
python main.py --demographic age
```

```
python main.py --demographic gender
```

```
python main.py --demographic cs_expertise
```

```
python main.py --demographic education
```

```
python main.py --demographic nlp_experience
```

Typical Workflow: 1. **Process raw data** (only needed if raw data changes): bash

```
python main.py --data-only
```

2. **Run main hypothesis tests:**

```
python main.py --hypothesis-only
```

3. **Run complete analysis** (recommended for first-time use):

```
python main.py
```

4. **Explore specific demographics** (as needed):

```
python main.py --demographic age
```

Dependencies: The script automatically checks for required packages (pandas, numpy, scipy, statsmodels, matplotlib, seaborn, openpyxl) and provides clear error messages if dependencies are missing.

9.7 Output Files and Locations

The analysis pipeline generates structured outputs organized by type and purpose, suitable for inclusion in thesis reporting and academic publication:

Data Files (in data/ directory)

File	Description
------	-------------

experiment_results_with	Main processed dataset with all computed metrics
-------------------------	--

File	Description
<code>_metrics.xlsx</code>	
<code>hypothesis_summary_table.csv</code>	Complete results table for all 16 hypotheses (coefficients, p-values, significance)
<code>participant_level_aggregates.csv</code>	Participant-level summary statistics (mean RAIR, RSR, accuracy per participant)
<code>within_subjects_comparisons_faithfulness.csv</code>	Paired comparison results for faithful vs. unfaithful conditions
<code>within_subjects_comparisons_modelsize.csv</code>	Paired comparison results for big vs. small model conditions

Plot Directories

plots/general/ - Main analysis visualizations (24 plots): - RAIR/RSR by faithfulness and model size - Accuracy distributions and per-question analyses - Confidence change visualizations - Plausibility ratings and relationships - Agreement and reliance scatter plots

plots/demographic_analyses/ - Demographic-specific outputs: - `age_plots/` - Age group analysis visualizations - `gender_plots/` - Gender analysis visualizations - `cs_expertise_plots/` - CS/AI expertise analysis outputs - `education_level_plots/` - Education level analysis outputs - `nlp_experience_plots/` - NLP experience analysis outputs

Each demographic subdirectory contains box plots, violin plots, scatter plots, and regression visualizations specific to that demographic variable.

9.8 Statistical Methodology

The analysis pipeline implements rigorous statistical methods appropriate for the within-subjects experimental design.

Within-Subjects Design

The user study uses a **repeated measures (within-subjects)** design, where each participant encounters multiple conditions: - Both faithful and unfaithful explanations (across different trials) - Both big and small model sizes (counterbalanced across forms A and B) - Multiple trials per condition

This design increases statistical power by controlling for individual differences, but requires special handling of correlated observations.

Cluster-Robust Standard Errors

Why Cluster-Robust SEs are Necessary: - Observations from the same participant are correlated (not independent) - Standard regression assumes independence, leading to underestimated standard errors - Underestimated SEs produce inflated t-statistics and

false positives - Cluster-robust SEs account for within-participant correlation, providing valid inference

Implementation: - All regression models cluster standard errors by participant ID - Uses statsmodels with `cov_type='cluster'` and `groups=participant_id` - Produces valid p-values and confidence intervals for repeated measures data

Effect Size Calculations

For Paired Comparisons: - Cohen's d (paired): Used when differences are normally distributed - Formula: $d = \text{mean_diff} / \text{SD_diff}$ - Interpretation: Small (0.2), Medium (0.5), Large (0.8)

- **Rank-biserial correlation:** Used for non-normal differences (Wilcoxon test)
 - Measures association strength for non-parametric comparisons

For Regression Models: - Coefficients represent change in outcome per unit change in predictor - Odds ratios for logistic regression (binary outcomes) - Standardized coefficients for continuous outcomes

Normality Testing

Before selecting parametric vs. non-parametric tests, the pipeline performs: - **Shapiro-Wilk tests** on continuous variables (overall and by groups) - **Visual inspection** via Q-Q plots and histograms - Appropriate test selection based on normality assumptions

This methodological rigor ensures that the analysis pipeline produces valid, reproducible results suitable for publication in academic research.

10. Further Developments and Future Work

While LET provides a comprehensive platform for generating and evaluating explanations, there are several areas where the prototype could be extended and improved. This section outlines potential enhancements and current limitations that represent opportunities for future development.

10.1 User Feedback and Ratings System

Current State: LET collects user feedback on explanations through a 1-5 rating scale, allowing users to rate the quality and usefulness of generated explanations. Ratings are stored in the database alongside explanation metadata, timestamps, and user identifiers. However, the current implementation has significant limitations:

- **Limited Feedback Mechanism:** Only numeric ratings (1-5 scale) are collected, without qualitative feedback or detailed comments

- **Underutilized Data:** While ratings are collected and stored, they are not currently analyzed, visualized, or integrated into the evaluation framework
- **No Aggregation:** There is no dashboard or analysis pipeline to aggregate ratings across explanations, models, or datasets
- **Missing Correlations:** Ratings are not correlated with objective metrics such as faithfulness scores, plausibility metrics, or user behavioral data (RAIR, RSR)

Potential Enhancements: - **Qualitative Feedback:** Add text fields for users to provide written comments explaining their ratings, enabling richer understanding of user perceptions - **Ratings Analysis Pipeline:** Integrate ratings analysis into the result scripts to correlate subjective ratings with objective metrics, enabling investigation of whether higher-rated explanations correspond to higher faithfulness or plausibility scores - **Dashboard Visualizations:** Create visualization dashboards showing average ratings by model, explanation type, dataset, and other dimensions, facilitating quick identification of patterns - **Feedback Loop:** Use ratings to identify patterns and improve explanation generation strategies, potentially training models to generate explanations that users find more helpful

User Studies Readiness: The ratings system can be extended to support comprehensive user studies, making it a valuable tool for research on human-AI interaction. Potential extensions include:

- **Structured Questionnaires:** Extend beyond simple 1-5 ratings to include structured questionnaires with multiple dimensions (e.g., clarity, helpfulness, trustworthiness, completeness)
- **Multi-dimensional Rating Scales:** Implement Likert scales for different aspects of explanation quality, allowing researchers to understand which dimensions matter most to users
- **Longitudinal Tracking:** Track user feedback over time to understand how perceptions change with experience or exposure to different explanation types
- **Integration with Behavioral Metrics:** Combine ratings with behavioral metrics (RAIR, RSR, accuracy changes) collected during studies to provide a comprehensive view of explanation effectiveness
- **Export Functionality:** Enable export of ratings data in formats suitable for analysis in external statistical tools (R, SPSS, Python), facilitating integration with existing research workflows

10.2 Evaluation Framework Extensions

Current State: LET implements the LExT framework for evaluating faithfulness and plausibility, providing a solid foundation for explanation quality assessment.

Potential Enhancements: - **Additional Metrics:** Integrate other evaluation metrics such as BLEU, ROUGE, semantic similarity scores, or task-specific metrics that may be relevant for particular domains

10.3 User Interface and Experience

Current Features: The LET interface provides basic functionality for dataset management, classification, and explanation viewing, but there are opportunities for enhancement.

Potential Enhancements: - **Advanced Filtering and Search:** Enable filtering of results by metrics, model, dataset, or date range, and add search functionality for explanations and predictions, facilitating exploration of large result sets - **Export Functionality:** Add export capabilities for results as PDF reports, LaTeX tables, or structured JSON/CSV formats, enabling integration with research papers and external analysis tools - **Progress Tracking:** Implement real-time progress bars and status updates for batch operations, improving user experience during long-running tasks - **Collaboration Features:** Add functionality to share runs and datasets with team members, and enable comments and annotations on explanations, supporting collaborative research workflows

Appendix A: Complete Prompt Templates

This section provides the complete set of prompt templates for all supported datasets. For space considerations, the main document included only representative examples (IMDB, ECQA, Deceptive Opinion). Below are the prompts for the remaining datasets.

A.1 CaseHOLD Prompts

Self-Explanation and Classification

Assume you are a legal advisor.

Given the following legal case context and five potential holdings, select the correct legal holding and explain your reasoning in 2-3 sentences.

Context: {context}

Holdings:

```
{holding[0]}
{holding[1]}
{holding[2]}
{holding[3]}
{holding[4]}
```

Format your answer as:

Answer: <Your Choice>

Explanation: <your explanation here>

Post-Hoc Explanation

Assume you are a legal advisor.

Explain this legal case holding selection in simple terms:

Context: {context}
Selected Holding: {selected_holding}
Confidence: {score}%

Focus on the legal reasoning and key facts.
Keep explanation under 3 sentences.

A.2 PubMedQA Prompts

Self-Explanation and Classification

Assume you are a medical advisor.

Based on the following biomedical question and context from a research abstract,
answer the question and explain your reasoning in 2-3 sentences.

Question: {question}

Context: {context}

Format your answer as:
Answer: <yes / no / maybe>
Explanation: <your explanation here>

Post-Hoc Explanation

Assume you are a medical advisor.

Explain this biomedical question answering result in simple terms:

Question: {question}
Context: {context}
Answer: {answer} ({score}% confidence)

Focus on the key medical evidence and reasoning.
Keep explanation under 3 sentences.

A.3 SNARKS Prompts

Self-Explanation and Classification

You are analyzing text for sarcasm detection.

Given the following text, determine if it is sarcastic and explain your reasoning in 2-3 sentences.

Text: {text}

Format your answer as:

Answer: <sarcastic / not sarcastic>
Explanation: <your explanation here>

Post-Hoc Explanation

You are analyzing text for sarcasm detection.

Explain this sarcasm detection result in simple terms:

Text: {text}
Prediction: {label} ({score}% confidence)

Focus on tone, context, and linguistic cues.
Keep explanation under 3 sentences.

Appendix B: Dataset Schema Specifications

This section documents the expected schema for custom dataset uploads.

B.1 General Requirements

All datasets must be provided in CSV format with UTF-8 encoding. The first row should contain column headers. Missing values should be left empty or marked as NaN.

B.2 Sentiment Analysis (IMDB-style)

Column	Type	Required	Description
text	string	Yes	Review or input text
label	string	Yes	Sentiment label (positive/negative)
split	string	No	train/test/validation

B.3 Multiple Choice (ECQA, CaseHOLD-style)

Column	Type	Required	Description
question	string	Yes	Question text
choice_0	string	Yes	First answer option
choice_1	string	Yes	Second answer option
choice_2	string	Yes	Third answer option
choice_3	string	Yes	Fourth answer option
choice_4	string	Yes	Fifth answer option
answer	string	Yes	Correct answer (matches one of the choices)
explanation	string	No	Gold rationale (for plausibility evaluation)

B.4 Question Answering (PubMedQA-style)

Column	Type	Required	Description
question	string	Yes	Clinical or biomedical question
context	string	Yes	Supporting passage or abstract
answer	string	Yes	yes/no/maybe
long_answer	string	No	Gold explanation

B.5 Binary Classification (Deceptive Opinion, SNARKS-style)

Column	Type	Required	Description
text	string	Yes	Input text (review, tweet, etc.)
label	string	Yes	Binary label (truthful/deceptive, sarcastic/not sarcastic)
